University of Warsaw
Faculty of Economic Sciences

Krzysztof Wojdalski

Nr albumu: 310284

Microstructure-Aware Reinforcement Learning for High-Frequency Trading

Praca magisterska
na kierunku ECONOMICS

The thesis written under the supervision of
dr Pawel Sakowski

2025-05-03

Data Podpis opiekuna

Statement of the Supervisor on Submission of the Thesis

I hereby certify that the thesis submitted has been prepared under my supervision and I declare that it satisfies the requirements of submission in the proceedings for the award of a degree.

Date Signature of the Supervisor:

Oświadczenie autora (autorów) pracy

Data Podpis autora (autorów) pracy

Statement of the Author(s) on Submission of the Thesis

Aware of legal liability I certify that the thesis submitted has been prepared by myself and does not include information gathered contrary to the law.

I also declare that the thesis submitted has not been the subject of proceedings resulting in the award of a university degree.

Furthermore I certify that the submitted version of the thesis is identical with its attached electronic version.

Date Signature of the Author(s) of the thesis

Table of contents

1. Introduction **9**
1.1. Scope and Objectives of the Research . . . 10
1.1.1. Novelty of This Research . . . 10
1.1.2. Hypothesis . . . 11
1.1.3. Objectives of the Research . . . 11
1.2. Thesis Organization and Chapter Overview . . . 12

2. Literature Review **14**
2.1. Market Microstructure Evidence for LOB-Based Trading . . . 14
2.1.1. Bid-Ask Spread and Short-Horizon Serial Correlation . . . 14
2.1.2. Intraday Periodicity . . . 15
2.1.3. Price Discreteness and Quote Clustering . . . 15
2.1.4. Implications for This Thesis . . . 15
2.2. Market Microstructure Theory . . . 15
2.2.1. Adverse Selection and the Spread Decomposition . . . 16
2.2.2. Limit Order Book Mechanics . . . 16
2.2.3. Order Flow and Short-Term Price Prediction . . . 17
2.2.4. Fair Value Estimation from the Order Book . . . 17
2.2.5. Synthesis . . . 18
2.3. Why Reinforcement Learning for Trading . . . 18
2.4. Key Concepts from Reinforcement Learning . . . 19
2.4.1. Markov Decision Processes . . . 19
2.4.2. The Bellman Equation and Dynamic Programming . . . 19
2.4.3. Temporal Difference Learning . . . 20
2.4.4. Policy Gradient Methods . . . 20
2.4.5. Exploration-Exploitation Tradeoff . . . 20
2.4.6. Function Approximation and Deep RL . . . 21
2.4.7. Reward Shaping and Credit Assignment . . . 21
2.4.8. Sample Efficiency and Off-Policy Learning . . . 21
2.4.9. Non-Stationarity and Continuous Adaptation . . . 21
2.5. RL Applications in Trading Systems . . . 22
2.5.1. Early Applications and Proof-of-Concept . . . 22
2.5.2. From Discrete to Continuous Action Spaces . . . 22
2.5.3. Deep Reinforcement Learning Era . . . 23
2.5.4. Comparative Studies . . . 24
2.5.5. Structured Comparison of Representative Studies . . . 24
2.5.6. Challenges and Limitations . . . 25
2.5.7. Synthesis and Implications for This Thesis . . . 26

3. Machine Learning **27**
3.0.1. Supervised Learning . . . 27
3.0.2. Unsupervised Learning . . . 28
3.0.3. Semi-Supervised Learning . . . 28
3.0.4. Reinforcement Learning . . . 28
3.1. Algorithmic Trading Systems . . . 29
3.1.1. Components Relevant to This Thesis . . . 29
3.1.2. Scope Boundary . . . 31

3.1.3. Rule-Based Trading Strategies . . . 31
3.1.4. Trading based on Forecasts . . . 32
3.1.5. Training a Trading System on Labeled Data . . . 33
3.1.6. Direct Optimization of Performance . . . 33

4. Reinforcement Learning **35**
4.1. Components of a Reinforcement Learning System . . . 35
4.1.1. Environment . . . 35
4.1.2. State . . . 35
4.1.3. Action . . . 36
4.1.4. Policy . . . 36
4.1.5. Reward . . . 36
4.1.6. Value Function . . . 36
4.1.7. Applicability to Trading . . . 37
4.2. From Tabular Methods to TD3: An Algorithmic Lineage . . . 37
4.2.1. Value-Based Methods and Their Limitations in Continuous Control . 37
4.2.2. Policy Gradient Methods and the Actor-Critic Architecture . . . 38
4.2.3. DDPG: Deterministic Policies for Continuous Control . . . 38
4.2.4. TD3: Addressing the Instability of Deterministic Actor-Critic Methods 39
4.3. Classifying RL Algorithms . . . 40
4.3.1. Model-free vs Model-based . . . 40
4.3.2. Value-Based, Policy-Based, and Actor-Critic . . . 40
4.3.3. On-Policy vs Off-Policy . . . 41

5. Design of the Trading Agent **42**
5.1. Action Space . . . 42
5.1.1. Continuous vs Discrete Actions . . . 42
5.1.2. Implementation: Box Portfolio Action Space . . . 42
5.1.3. Action Constraints and Transaction Costs . . . 43
5.1.4. Action Smoothing and Exploration Noise . . . 43
5.2. State Space . . . 44
5.2.1. Design Requirements . . . 44
5.2.2. Feature Groups and Selection Rationale . . . 45
5.2.3. Observation Space and Normalization . . . 49
5.2.4. On the Exclusion of Raw LOB Columns . . . 49
5.3. Reward Function . . . 50
5.3.1. Why Raw Log-Return Is Insufficient . . . 50
5.3.2. Differential Sharpe Ratio . . . 51
5.3.3. Reward and Financial Return Are Not the Same . . . 51
5.4. Value Function . . . 52
5.4.1. Twin Critics . . . 52
5.4.2. Loss Function . . . 52
5.5. Policy . . . 53
5.6. Discount Factor . . . 53
5.7. Optimization Hyperparameters . . . 53
5.7.1. Learning Rates . . . 53
5.7.2. Weight Decay . . . 53
5.7.3. Scope . . . 54

6. Implementation of the Trading Agent **55**
6.1. Simulation Assumptions . . . 55
6.2. Data Preparation . . . 56
6.2.1. Input Format . . . 56
6.2.2. Preprocessing and Splitting . . . 56
6.2.3. Feature Normalization and Leakage Prevention . . . 57
6.2.4. Feature and Price Column Separation . . . 57
6.3. Implementation and Training Pipeline . . . 57
6.3.1. Configuration-Driven Design . . . 57
6.3.2. Feature Pipeline . . . 58
6.3.3. Training Loop . . . 58
6.3.4. Evaluation . . . 58

7. Empirical Evaluation and Performance Analysis **59**
7.1. Experiment Snapshot . . . 59
7.2. Statistical Validation . . . 59
7.2.1. Purpose and Scope of Statistical Validation . . . 59
7.2.2. Run-Level Sample Summary . . . 60
7.2.3. Descriptive Statistics and Uncertainty Estimates . . . 60
7.2.4. Comparative Validation Strategy for Thesis-Grade Results . . . 60
7.2.5. Statistical Risks and Interpretation Caveats . . . 61
7.3. Robustness Assessment . . . 61
7.3.1. Why Robustness Matters in This Thesis . . . 61
7.3.2. Current Evaluation Context . . . 62
7.3.3. Robustness Dimensions Considered in This Thesis . . . 62
7.3.4. Robustness Matrix for Experimental Reporting . . . 62
7.3.5. Interpreting Noisy Training Curves in Robustness Terms . . . 62
7.3.6. Methodological Limits of the Current Robustness Snapshot . . . 63
7.4. Performance Evaluation . . . 63
7.4.1. Selected Metrics from the Latest Completed Run . . . 63
7.4.2. Evaluation Metadata Snapshot . . . 63
7.4.3. Evaluation Plots . . . 63
7.4.4. Discussion: Reading Performance in the Context of State Design . . . 63

8. Conclusions and Future Work **65**
8.1. Summary of Findings . . . 65
8.2. Limitations and Future Research . . . 65
8.2.1. Order-Book Execution Realism Constraint . . . 65
8.2.2. Agent-State Feature Extensions . . . 66
8.2.3. Cross-Asset and Multi-Asset Extensions . . . 66
8.2.4. Generalization Across Asset Classes and Market Regimes . . . 66
8.2.5. Algorithm Comparison and Reward Ablation . . . 67
8.2.6. Evaluation Robustness . . . 67
8.3. Implications for Trading Systems . . . 67
8.4. Recommendations for Practitioners . . . 67
8.5. Conclusion . . . 67

Bibliography **69**

A. LOB Feature Inventory **73**

B. TD3 Hyperparameters **74**

C. Experiment Configuration Example **75**

D. TD3 Training Workflow Diagram **77**

E. TD3 Update Mechanics Diagram **77**

F. RL Algorithm Taxonomy **78**

G. TD3 Network Architecture **80**

H. Data Preparation Pipeline **80**

I. TD3 Training Pseudo-Code **81**

Abstract

This thesis studies whether deep reinforcement learning can produce robust, risk-adjusted trading performance in a realistic high-frequency setting. The work focuses on continuous portfolio exposure control from order-book-derived features and addresses practical constraints that often invalidate paper-trading results.

The empirical framework implements a TD3 agent on high-frequency AAPL data and evaluates it against benchmark strategies under explicit assumptions about transaction costs and execution. Performance is assessed with both financial metrics and statistical validation to separate genuine signal from noise and overfitting.

The main contribution is an end-to-end, reproducible pipeline that links market feature engineering, reward design, agent training, and robustness testing in a single evaluation framework for algorithmic trading research.

1. Introduction

Much of the existing reinforcement learning literature in trading relies on low-frequency market representations such as daily or intraday OHLCV bars. High-frequency trading, however, is governed by information embedded in the limit order book, where local liquidity, queue dynamics, spread changes, and event timing shape short-horizon opportunities and risks. This thesis examines whether a continuous-control reinforcement learning agent can use order-book-derived features to make economically meaningful trading decisions in a high-frequency single-asset setting.

High-frequency trading should therefore not be treated as a simple faster version of bar-based trading. The state observed by the agent is structurally different. At short horizons, aggregated price bars omit information that is central to execution-sensitive decision-making. Features derived from the order book, such as depth imbalance, spread variation, order-flow pressure, and inter-event timing, provide a more faithful description of local market conditions than coarse candlestick aggregates. A central concern of this thesis is whether such microstructure-aware state representations improve the suitability of reinforcement learning for this domain.

Reinforcement learning is a plausible modelling framework for this problem because trading is sequential and path dependent. Each action affects future exposure, portfolio evolution, and the set of subsequent decisions available to the agent. In high-frequency settings, this distinction matters. Industry deployments such as JPMorgan's execution engine (Mosic 2017) have applied RL to limit order book data for optimal execution — minimizing implementation shortfall on large orders given a known trade schedule, where the LOB state informs fill probability and spread cost rather than price direction. Directional exposure control is a harder problem: the reward signal at tick frequency is dominated by microstructure noise rather than genuine price movement, the LOB state must be transformed into features that carry predictive signal rather than liquidity estimates, and the agent must generalize across intraday regimes whose statistical properties can differ substantially from the training window.

This thesis investigates the use of reinforcement learning, with particular focus on TD3, for high-frequency single-asset trading based on order-book-derived information. The research is motivated not by a claim that artificial intelligence can universally solve trading, but by a

narrower and testable question: under controlled experimental conditions, can a continuous control agent using microstructure-aware features achieve economically meaningful out-of-sample performance relative to selected benchmarks?

The problem is challenging for several reasons. Financial reward signals are noisy and non-stationary, transaction costs can erase apparent profitability, and short-horizon strategies are highly sensitive to turnover, feature specification, and market frictions. In addition, strong empirical claims are difficult because favorable in-sample results may reflect data leakage, overfitting, or unrealistic execution assumptions rather than genuine decision quality.

Accordingly, the thesis emphasizes methodological discipline alongside model development. It treats feature engineering, chronological evaluation, reward-return separation, and robustness analysis as core parts of the research problem rather than secondary implementation details. In this setting, the question is not only whether an agent can learn, but whether the learning setup is sufficiently well specified to support credible empirical conclusions.

At the same time, the thesis does not claim to reproduce the full operational complexity of live high-frequency trading. The empirical framework is based on a controlled simulation environment rather than a complete execution engine with queue-priority modelling, latency competition, exact fill uncertainty, and full market impact. In particular, portfolio valuation relies on a mid-price-like execution proxy rather than fully executable bid/ask fills, and the agent is treated as price-taking rather than as a participant that alters the state of the order book. The contribution of the thesis should therefore be understood as methodological and empirical: it evaluates reinforcement learning agents in a microstructure-aware setting while making the simplifying assumptions explicit.

1.1. Scope and Objectives of the Research

1.1.1. Novelty of This Research

The primary novelty of this thesis is the application of **TD3** to **high-frequency trading with order-book-derived data** for continuous portfolio-exposure control. This differs from a large share of prior RL trading studies that focus on lower-frequency (often daily or bar-based) inputs and evaluate algorithms such as Q-learning, DQN, or PPO primarily in bar-level settings.

The distinguishing contribution here is the combined focus on:

1. **Algorithmic choice**: TD3 as the main continuous-control learner for exposure/position sizing.
2. **Data frequency and structure**: high-frequency order-book stream rather than only aggregated OHLCV bars.
3. **Feature space**: microstructure-oriented signals (depth/imbalance/spread context) in addition to standard indicator-style features.

This thesis therefore contributes by evaluating the RL trading problem in a setting where algorithm class, data granularity, and feature design are jointly aligned with high-frequency market microstructure. The targeted algorithm-data-frequency combination remains less represented in prior work and defines the core novelty claim tested in the empirical chapters.

1.1.2. Hypothesis

The research is based on the following hypotheses:

1. A TD3-based trading agent using order-book-derived high-frequency features can achieve stronger out-of-sample risk-adjusted performance than selected benchmark strategies under the assumptions of the simulated environment.
2. Within the order-book-based trading framework studied in this thesis, extending the state representation from basic snapshot features to a broader microstructure-aware feature set leads to improved out-of-sample performance under identical training and evaluation conditions.
3. The empirical performance of reinforcement learning trading agents is materially sensitive to feature specification, reward design, and transaction-cost assumptions, which makes robustness analysis necessary for credible conclusions.

1.1.3. Objectives of the Research

The primary research goal is to evaluate Reinforcement Learning-based algorithms for high-frequency single-asset trading using order-book-derived data. The research objectives are structured in three interconnected phases:

1. **Design and Implementation**
    - Develop Reinforcement Learning trading agents for high-frequency order-book trading applications

- Construct a testing framework to evaluate agent performance

2. **Testing and Evaluation**
   - Assess agent performance on out-of-sample data against established benchmarks with use of established and novel risk-return metrics
   - Analyze robustness under varying market conditions
   - Determine whether agents can outperform selected benchmark strategies under the defined experimental assumptions
3. **Analysis and Implications**
   - Compare alternative order-book-derived feature sets within a common reinforcement learning training and evaluation framework
   - Assess whether broader microstructure-aware feature sets improve out-of-sample results relative to simpler snapshot-based state representations
   - Assess the practical relevance and limitations of the approach under simplified execution assumptions
   - Identify the binding constraints on out-of-sample performance — whether they originate in execution modeling simplifications, feature set coverage, or training data volume — to prioritize specific extensions for future work

1.2. Thesis Organization and Chapter Overview

This thesis is organized to move from problem formulation and literature motivation to agent design, implementation, and empirical evaluation in a microstructure-aware trading setting.

Chapter 1 introduces the problem context, defines the research scope, and presents the methodological framework used throughout the thesis.

Chapter 2 reviews the literature most directly relevant to the present research question. It focuses on market microstructure evidence motivating order-book-derived features, the structural case for reinforcement learning in trading, and prior RL trading applications with attention to action-space design, transaction-cost modeling, and evaluation rigor.

Chapter 3 places reinforcement learning within the broader landscape of machine learning and algorithmic trading system design. It contrasts supervised, unsupervised, and reinforcement learning approaches, then situates the trading agent within a larger trading-system architecture and alternative decision paradigms.

Chapter 4 develops the reinforcement learning foundations required for the thesis, with particular emphasis on the progression from earlier methods to TD3 and on the distinctions that matter for continuous-control trading problems.

Chapter 5 presents the design of the trading agent itself. It defines the continuous action space, the microstructure-aware state representation, the reward formulation, and the main policy and value-function design choices used in the empirical system.

Chapter 6 describes the implementation of the research pipeline, including data preparation, configuration-driven feature engineering, code structure, and the TD3-based training workflow used to produce reproducible experiments.

Chapter 7 reports the empirical results. It combines experiment snapshots, performance evaluation, statistical validation, and robustness assessment to separate promising outcomes from noise, overfitting, or fragile conclusions.

Chapter 8 synthesizes the findings, discusses the main limitations of the current framework, and draws implications for future research and practical trading-system development.

2. Literature Review

This chapter surveys the academic foundations relevant to reinforcement learning for high-frequency trading. It is organized in two parts. The first part reviews market microstructure — the empirical evidence and theoretical mechanisms that govern short-horizon price formation in limit order book markets. This establishes the financial environment in which the agent operates and motivates the feature engineering choices made in Chapter 5. The second part positions reinforcement learning as the appropriate methodological framework for this problem, surveys the key concepts that matter in a trading context, and reviews prior RL trading applications with attention to action-space design, transaction-cost modeling, and evaluation rigor. The survey concludes by identifying the specific gap — the joint alignment of LOB-derived features, continuous control, and microstructure-aware state design — that this thesis addresses.

2.1. Market Microstructure Evidence for LOB-Based Trading

At intraday and tick-level timescales, financial markets exhibit systematic structural patterns that are distinct from the macro-level anomalies studied in the asset pricing literature. These patterns arise not from investor irrationality or information asymmetry in the fundamental sense, but from the mechanics of how orders interact in a limit order book: the strategic behavior of market makers, the clustering of institutional activity, and the discrete price grid imposed by exchange rules. Understanding these patterns is a prerequisite for designing an HFT trading agent, since they define the informational environment the agent operates in and directly motivate the feature set used in this thesis.

2.1.1. Bid-Ask Spread and Short-Horizon Serial Correlation

Roll (1984) demonstrated that bid-ask spreads induce negative serial correlation in transaction prices at very short horizons, as prices bounce between bid and ask quotes. This microstructure noise distinguishes observed transaction prices from the latent efficient price and creates a predictable pattern exploitable only by agents that operate at tick frequency and account for spread costs explicitly. In this thesis, spread in basis points and the spread ratio feature directly encode the cost of crossing the spread, conditioning the agent's position-sizing decisions on the current liquidity environment.

2.1.2. Intraday Periodicity

Wood, McInish, and Ord (1985) and Harris (1986) documented systematic intraday patterns in volume, volatility, and returns: a pronounced U-shape in trading activity and spread width, with elevated liquidity demand at market open and close and a quiet midday period. These patterns reflect institutional trading schedules and liquidity provision dynamics rather than information arrival, and they are stable enough across days to serve as a conditioning signal for a trading agent. The hour-of-day sine and cosine encodings in the observation vector are motivated directly by this evidence — the agent needs to distinguish early-session conditions (wide spreads, high adverse selection) from midday conditions (tighter spreads, lower activity) when sizing positions.

2.1.3. Price Discreteness and Quote Clustering

Harris (1991) showed that prices cluster at round numbers and psychologically attractive levels, creating discontinuities in the order book that affect optimal order placement and execution. In a 10-level LOB, price clustering manifests as uneven spacing between price levels — a pattern captured by the book convexity feature, which measures the curvature of the price ladder on each side. Awareness of price clustering is also relevant to the spread ratio feature, since clusters create temporary spread compression and expansion that deviates from the recent average.

2.1.4. Implications for This Thesis

These three findings collectively establish that short-horizon price dynamics are structured, not random. The bid-ask bounce motivates spread-aware features and reward design; intraday periodicity motivates temporal context features; price discreteness motivates curvature features of the order book. Rather than treating the limit order book as a transparent window onto an efficient price, the empirical evidence supports modeling it as a strategic environment with predictable mechanical regularities — which is precisely the setting in which a reinforcement learning agent can learn to act systematically.

2.2. Market Microstructure Theory

The empirical regularities described in the preceding section are grounded in a theoretical literature that models the strategic interactions between informed traders, uninformed liquidity

demanders, and market makers in a limit order book. This section reviews the theoretical frameworks that directly underpin the feature design choices made in Chapter 5.

2.2.1. Adverse Selection and the Spread Decomposition

Glosten and Milgrom (1985) established that the bid-ask spread reflects a market maker's protection against trading with informed counterparties. In their model, the market maker faces two types of order flow: uninformed liquidity traders, whose trades are uncorrelated with future price changes, and informed traders, who possess private information about fundamental value. Setting a positive spread allows the market maker to profit from uninformed flow and recoup losses on informed flow. The spread therefore decomposes into an adverse selection component — compensation for information risk — and an inventory or order processing component.

This decomposition has a direct implication for trading signal design. A narrowing spread signals reduced adverse selection risk; a widening spread signals elevated informed activity. The spread features in Chapter 5 — spread in basis points and the spread ratio — are motivated by this theoretical framework. Their predictive content derives from what the spread reveals about the current composition of order flow, not merely from the mechanical cost of execution.

Kyle (1985) formalized the price impact of informed trading through the lambda parameter, which measures the sensitivity of price changes to order flow. In Kyle's continuous-time model, the informed trader optimally splits their order to minimize price impact, and the market maker sets prices to break even against order flow. The resulting relationship between order imbalance and price change is linear, with slope λ that depends on the ratio of informed to uninformed volume. The order flow features in Chapter 5 — OFI and signed trade flow — can be understood as empirical proxies for Kyle's order imbalance variable, with the bid-ask slope features measuring the depth analogue of λ across levels of the order book.

2.2.2. Limit Order Book Mechanics

Glosten (1994) developed the theory of the open limit order book as a competitive equilibrium, showing that a full LOB can sustain itself as a market mechanism under adverse selection. The key insight is that the price schedule embedded in the LOB — the sequence of bid and ask quotes at increasing depths — reflects the cumulative information content of orders at

each quantity level. Deeper levels of the book attract less informed order flow but carry higher execution uncertainty for those posting limit orders.

This theoretical result motivates the multi-level features used in Chapter 5. The book is not a single price but a price schedule, and the shape of that schedule — measured by depth ratios, slopes, and convexity features — encodes information about how informed the marginal liquidity provider expects the market to be at each quantity level.

2.2.3. Order Flow and Short-Term Price Prediction

Cont, Kukanov, and Stoikov (2014) provide the most direct empirical and theoretical treatment of the relationship between LOB events and short-term price changes. Their framework decomposes all LOB events into limit order arrivals, cancellations, and market orders, and defines order flow imbalance (OFI) as the net directional pressure these events exert on the best bid and ask queues. In a sample of NYSE stocks, they find a linear relationship between OFI and mid-price changes over intervals of one second to one minute, with OFI explaining 60–80% of short-term price variation. Importantly, the relationship is contemporaneous: OFI at time t predicts price changes at time t, not a lagged effect, which is consistent with the efficient incorporation of order book information into prices.

The theoretical channel is straightforward: when buy-side pressure dominates (positive OFI), market makers raise quotes to attract sell-side flow and reduce inventory risk, mechanically pushing the mid-price upward. The OFI feature in this thesis operationalizes this mechanism directly.

2.2.4. Fair Value Estimation from the Order Book

Stoikov (2018) derives the microprice as the theoretically optimal estimate of the next transaction price given the current state of the top-of-book queues. The key observation is that the mid-price — the arithmetic average of best bid and ask — is an inadequate fair value estimate when the queues are imbalanced, because the side with the thinner queue is more likely to be depleted first by incoming market orders, pulling the next transaction price toward the opposite side. The microprice corrects for this by volume-weighting the best quotes:

$$\text{Microprice} = \frac{V_0^{ask} \cdot P_0^{bid} + V_0^{bid} \cdot P_0^{ask}}{V_0^{bid} + V_0^{ask}}$$

When $V_0^{bid} \gg V_0^{ask}$, the microprice is pulled toward the ask — the side more likely to move. The microprice divergence from mid captures this adjustment as a stationary signal and serves as the primary fair value feature in Chapter 5.

2.2.5. Synthesis

Together these four theoretical frameworks establish the informational structure of the limit order book. The spread encodes adverse selection risk (Glosten and Milgrom 1985); price impact and order imbalance are related through Kyle's lambda (Kyle 1985); the shape of the full book encodes the information content at each depth level (Glosten 1994); and OFI is the dominant contemporaneous predictor of short-term price changes (Cont, Kukanov, and Stoikov 2014). The microprice (Stoikov 2018) provides a theoretically optimal fair value estimate that conditions on the current queue imbalance.

These mechanisms are not independent: they all reflect the same underlying process of information aggregation through order flow. The feature set in Chapter 5 is designed to measure each mechanism from a distinct angle, providing the agent with a comprehensive description of the information environment it operates in.

2.3. Why Reinforcement Learning for Trading

Before surveying applications, it is worth establishing why reinforcement learning is a natural fit for algorithmic trading rather than an arbitrary methodological choice. The argument rests on three structural properties of the trading problem.

Unknown market dynamics. Model-based approaches require an explicit transition model — a specification of how the environment responds to actions. In financial markets, this model is unknown and non-stationary: price dynamics depend on the aggregate behavior of all participants, which changes continuously. Model-free RL methods learn directly from experience without requiring a transition model, making them the appropriate class for this setting (Sutton and Barto 2018).

Off-policy learning from historical data. Online experimentation in live markets is constrained by capital, risk, and regulatory requirements. The alternative is to learn from historical order book data — a fixed dataset of past transitions. Off-policy algorithms such as TD3 (Fujimoto, Hoof, and Meger 2018) can train on stored experience rather than requiring

fresh environment interaction at every update, making them sample-efficient in the data-limited financial setting.

Continuous position sizing. Trading decisions are naturally continuous: the agent should be able to express any degree of conviction through the size of its position, not just a binary buy/sell signal. Value-based methods such as DQN (Mnih et al. 2015) require a discrete action space and cannot represent fractional positions without reintroducing a grid approximation. Deterministic actor-critic methods operate natively in continuous action spaces and are therefore more appropriate for position-sizing tasks.

Together these three properties point toward a model-free, off-policy, continuous-action algorithm — which is the class to which TD3 belongs. The formal algorithmic properties of TD3 are derived in Chapter 4. The present section is concerned with how the broader RL literature has been applied to trading and what that literature implies for the design choices made in this thesis.

2.4. Key Concepts from Reinforcement Learning

This subsection identifies the reinforcement learning concepts that are most important for interpreting the finance literature reviewed in this chapter. To avoid duplication with the dedicated theory chapter, formal definitions, equations, and algorithm taxonomies are presented in Chapter 4; here the focus is on conceptual relevance to trading problems and the corresponding literature.

2.4.1. Markov Decision Processes

The Markov Decision Process (MDP) framework (Bellman 1957; Puterman 1994) provides the formal language for reinforcement learning. Portfolio decisions are sequential and path-dependent, which makes MDP-based reasoning the natural starting point. At the same time, market regime shifts, structural breaks, and partial observability mean that financial environments only approximately satisfy standard MDP assumptions. The formal definitions of states, actions, rewards, and value functions are developed in Chapter 4.

2.4.2. The Bellman Equation and Dynamic Programming

Bellman's principle of optimality remains central because it motivates value-based reasoning about long-run consequences of trading actions Bellman (1957). Dynamic-programming

methods are typically not directly applicable in live trading settings because market transition dynamics are unknown and unstable, but the Bellman perspective still underpins how modern RL algorithms estimate future reward and compare actions.

2.4.3. Temporal Difference Learning

Temporal difference learning Sutton (1988) is particularly relevant to finance because it supports incremental learning from experience without a full model of market dynamics. Q-learning Watkins (1989) is especially influential in the literature because its off-policy nature aligns with a common financial setting: learning from historical trajectories while evaluating policies that were not necessarily executed in real time.

2.4.4. Policy Gradient Methods

Policy gradient methods became important for trading research because many portfolio problems are naturally formulated in continuous action spaces (e.g., position sizing or allocation weights). Foundational contributions such as REINFORCE Williams (1992) and the policy gradient theorem Sutton et al. (2000) support actor-critic designs that are better suited than tabular methods to continuous control and high-dimensional observations.

2.4.5. Exploration-Exploitation Tradeoff

The exploration-exploitation tradeoff is especially costly in financial applications because exploratory actions can incur immediate losses and transaction costs, while under-exploration can prevent discovery of profitable policies. As emphasized in Sutton and Barto (2018), this is a core RL problem; in trading, it often motivates conservative exploration schemes and risk-sensitive reward designs such as those discussed by Moody and Saffell (1998).

In market settings, this tradeoff is amplified by non-stationarity: strategies that exploit historical regularities can decay quickly after regime changes, yet excessive exploration can destroy performance through churn and slippage. The literature therefore tends to favor controlled exploration schedules, stability-oriented algorithms, and evaluation protocols that separate genuine adaptation from overfitting.

2.4.6. Function Approximation and Deep RL

Financial state representations are typically high dimensional, making tabular methods impractical and motivating function approximation. The deep RL advances popularized by Mnih et al. (2015) accelerated finance applications (e.g., Deng et al. (2017)), but they also amplified known stability issues in off-policy learning with function approximation analyzed by Tsitsiklis and Van Roy (1997). This is one reason the thesis adopts TD3 Fujimoto, Hoof, and Meger (2018), which was designed to improve stability in continuous-control settings.

2.4.7. Reward Shaping and Credit Assignment

Reward specification is a central design decision in trading RL because it determines what the agent actually optimizes. The reward-shaping literature Ng, Harada, and Russell (1999) is therefore directly relevant to finance, where practitioners often combine dense learning signals with risk-aware objectives (e.g., Sharpe-style metrics, drawdown penalties, and transaction-cost-aware rewards). Consistent with this literature, the thesis uses log-return and Differential Sharpe Ratio-based signals to align optimization with risk-adjusted performance goals.

2.4.8. Sample Efficiency and Off-Policy Learning

Sample efficiency is unusually important in finance because historical data is finite and online experimentation is expensive. Off-policy methods, importance sampling ideas Precup, Sutton, and Singh (2000), and replay-based reuse of experience Lin (1992) are therefore more than implementation details: they are methodological enablers for learning from historical market data under limited experimentation budgets.

2.4.9. Non-Stationarity and Continuous Adaptation

Non-stationarity is the main reason why direct transfer of classical RL results to financial markets is difficult: market dynamics, volatility regimes, and participant behavior evolve over time. Work on continual adaptation and meta-learning discussed in Sutton and Barto (2018) and finance-oriented discussions such as Deng et al. (2017) motivates the thesis emphasis on robust feature engineering, conservative optimization, and out-of-sample temporal validation. The formal RL machinery used to operationalize these choices is developed in Chapter 4 and then specialized for the trading agent in Chapters 5-7.

Accordingly, claims that RL "works" in trading should be interpreted as context-dependent:

RL methods can produce useful policies when the learning objective, state representation, and validation design are aligned with market frictions and regime uncertainty.

2.5. RL Applications in Trading Systems

The application of reinforcement learning to financial trading has evolved from theoretical proposals to practical implementations across diverse market segments. This section surveys the literature on RL-based trading systems with a focus on methodology, action space design, and evaluation rigor.

To keep conclusions methodologically defensible, this review distinguishes between: (i) proof-of-concept evidence from backtests under simplified assumptions, and (ii) stronger evidence that includes robust out-of-sample design, realistic cost modeling, and variance reporting across repeated runs.

2.5.1. Early Applications and Proof-of-Concept

Moody and Saffell (1998) presented the first comprehensive application of RL to trading, introducing recurrent reinforcement learning for optimizing trading systems. His approach incorporated four design choices that remain influential: the Differential Sharpe Ratio as a risk-adjusted reward, a recurrent network architecture for temporal state encoding, direct policy optimization rather than a predict-then-trade pipeline, and explicit transaction cost modeling. Moody demonstrated that RL could outperform supervised learning baselines on S&P 500 futures, establishing feasibility of the approach and showing that optimizing risk-adjusted return directly was often superior to optimizing intermediate predictions.

Moody and Saffell (2001) extended this framework to multi-asset portfolios, validating that RL-optimized policies could generate positive risk-adjusted returns after transaction costs — a stringent test of practical viability.

2.5.2. From Discrete to Continuous Action Spaces

Early value-based approaches applied Q-learning with discrete buy/sell/hold actions to equity and futures markets (Neuneier 1998; J. W. Lee 2001). These studies established that RL could learn profitable patterns from historical data, but also identified persistent problems: sparse rewards, delayed credit assignment, and out-of-sample performance degradation. More

fundamentally, discrete action spaces force an artificial granularity on position sizing — a coarse grid loses control precision while a fine grid reintroduces the curse of dimensionality.

Gold (2003) and Dempster and Romahi (2002) demonstrated that policy gradient methods with continuous action spaces addressed this limitation directly. Continuous portfolio weights proved more natural for allocation tasks and policy gradient methods achieved better risk-adjusted returns with fewer training episodes, establishing the case for actor-critic approaches in trading.

2.5.3. Deep Reinforcement Learning Era

The introduction of deep function approximators to RL trading extended applicability to high-dimensional state spaces.

Xiong et al. (2018) and Deng et al. (2017) adapted DQN for equity and cryptocurrency trading, demonstrating that convolutional networks could extract useful features from raw price data. However, both studies noted high variance across runs — a characteristic instability of value-based methods under noisy financial rewards.

Jiang, Xu, and Liang (2017) proposed the portfolio-vector-memory (PVM) architecture, combining policy gradients with recurrent networks for cryptocurrency portfolio management. Their fully differentiable end-to-end design incorporated current holdings into the state and penalized transaction costs explicitly, outperforming Markowitz mean-variance allocation in backtests across 12 cryptocurrencies.

Liang et al. (2018) applied DDPG to stock trading and conducted ablation studies showing that: (i) DDPG outperformed DQN for continuous position sizing, (ii) Sharpe-based rewards improved out-of-sample generalization, and (iii) transaction cost modeling was critical for realistic performance assessment.

Majidi, Shamsi, and Marvasti (2024) provided a focused TD3 study on single-asset trading for AMZN and BTC using daily data and a continuous action space in $[-1, 1]$. Their minimal state representation — lagged close-return windows rather than large technical-indicator sets — reported improved return and Sharpe ratio for TD3 versus discrete variants and buy-and-hold, though results were mixed across assets. This study is best interpreted as evidence that continuous action sizing can be beneficial under realistic frictions, not as definitive proof of robust alpha across market regimes.

Kabbani and Duman (2022) proposed a TD3-based framework incorporating account state (cash and holdings), price features, technical indicators, and optional FinBERT-derived news sentiment. Richer state designs improved both return and Sharpe ratio relative to a close-price-only baseline, with a final 10-stock evaluation reporting a Sharpe ratio of 2.68. Methodologically the paper supports the practical value of feature enrichment in continuous-action actor-critic trading, while also illustrating how performance claims depend strongly on evaluation protocol choices.

Wang (2019) and Yang (2020) demonstrated PPO in cryptocurrency and high-frequency trading settings respectively, noting superior training stability relative to DDPG at the cost of lower sample efficiency. Yang (2020) in particular highlighted the scalability challenges of training stable policies on millisecond-resolution LOB data, requiring extensive hyperparameter tuning.

W. Zhang (2020) integrated attention mechanisms with deep RL for stock trading, enabling the agent to focus on relevant features dynamically. The attention weights provided partial interpretability — visualizing which indicators influenced trading decisions at each timestep — partially addressing the black box critique of deep RL systems.

2.5.4. Comparative Studies

Shen (2021) benchmarked DQN, DDPG, PPO, TD3, and SAC across asset classes, finding that off-policy methods (DDPG, TD3, SAC) learned faster than on-policy alternatives, with TD3 and SAC exhibiting the lowest variance across random seeds. No algorithm dominated across all markets; best choice depended on reward noise and action space dimensionality.

López de Prado (2018) compared deep RL against traditional quantitative strategies (momentum, mean reversion, pairs trading), noting that RL exhibited superior adaptability during regime changes while traditional strategies offered better interpretability. Hybrid approaches combining domain knowledge with RL often outperformed either in isolation.

2.5.5. Structured Comparison of Representative Studies

Study	Market / Task	State Variables	Action Space	Reward / Objective	Main Caveat
Moody & Saffell (1998)	Futures trading	Price-derived signals + recurrent state	Continuous position control	Differential Sharpe Ratio	Limited modern reproducibility standards
Jiang et al. (2017)	Crypto portfolio allocation	Price tensor + previous portfolio weights	Continuous portfolio weights	Commission-aware portfolio growth	Backtest dependence on selected period and universe
Liang et al. (2018)	Stock portfolio management	Market features + macro variables	Continuous (DDPG)	Return with friction-aware setup	Sensitivity to architecture and hyperparameters
Majidi et al. (2024)	Single-asset AMZN/BTC	Close-return sliding window (autoregressive)	Continuous $[-1, 1]$	Log-return with transaction cost	Mixed performance vs. buy-and-hold across assets
Kabbani & Duman (2022)	Multi-asset stock trading	Cash + holdings + close + indicators + sentiment	Continuous (TD3)	Portfolio return / Sharpe-oriented	Sharpe likely optimistic under strict frozen-policy evaluation
Shen et al. (2021)	Cross-algorithm benchmarking	Task-dependent feature sets	Mixed (discrete + continuous)	Comparative return / risk metrics	Result portability depends on data and protocol harmonization

2.5.6. Challenges and Limitations

Overfitting and Generalization

Krauss, Do, and Huck (2017) demonstrated that many published RL trading strategies fail to generalize out-of-sample, attributing this to high noise-to-signal ratios in financial data, limited training data relative to model complexity, and non-stationarity that invalidates training-period patterns. The implication is that evaluation methodology — strict temporal splits, multi-seed reporting, realistic cost assumptions — matters as much as algorithm choice.

Reward Specification

Cong (2020) argues that reward engineering is critical and often under-examined. Misspecified rewards can lead to metric gaming (optimizing Sharpe by avoiding trades), ignoring tail risks, and misalignment with actual investor preferences. The Differential Sharpe Ratio (Moody and Saffell 1998) addresses some of these concerns by optimizing risk-adjusted return directly, but does not fully resolve the tension between reward simplicity and the richness of real trading objectives.

2.5.7. Synthesis and Implications for This Thesis

The literature collectively supports several design choices made in this thesis:

1. **Continuous control methods** (TD3) are advantageous for position sizing, confirmed across multiple studies (Liang et al. 2018; Majidi, Shamsi, and Marvasti 2024; Kabbani and Duman 2022).
2. **Risk-adjusted rewards** improve generalization but must be paired with evaluation on realized metrics, not just the training objective.
3. **Transaction cost modeling** is non-negotiable for realistic assessment.
4. **LOB-based state representations** are underexplored in the literature — most studies use OHLCV or candlestick features. The present thesis addresses this gap by constructing a microstructure-grounded observation vector from 10-level order book data.
5. **Evaluation rigor** — strict train/validation/test splits and multi-seed dispersion reporting — is as important as algorithm choice, given the evidence on overfitting in financial RL.

3. Machine Learning

In quantitative trading, the main methodological choice is not simply whether to use machine learning, but how to formulate the decision problem. The same market data can support very different tasks: forecasting future returns, classifying market conditions, extracting latent structure, or selecting actions that directly determine portfolio exposure. These formulations are not interchangeable. They imply different objectives, different forms of supervision, and different links between model output and realized trading performance.

The purpose of the present chapter is therefore not to survey machine learning in generic terms, but to position reinforcement learning relative to other paradigms used in quantitative trading. This comparison matters because prediction-based approaches and sequential control approaches solve different problems. A forecasting model produces a signal that must later be translated into trades, whereas a reinforcement learning agent is trained to choose actions directly under transaction frictions, portfolio constraints, and delayed feedback.

3.0.1. Supervised Learning

Supervised learning trains a model on labeled input-output pairs $(X_i, Y_i)_{i=1}^n$ to minimize a loss function $\mathcal{L}(f_\theta(X), Y)$ over a held-out sample. In trading, supervised approaches have been widely applied to return prediction, directional classification, and volatility forecasting — tasks where historical outcomes provide a natural training signal (Krauss, Do, and Huck 2017; Z. Zhang, Zohren, and Roberts 2019).

The approach has a structural limitation in trading contexts: optimizing the forecast objective and optimizing the trading decision are two separate steps with potentially misaligned objectives. A model that minimizes mean squared error on next-period returns may not produce positions that maximize risk-adjusted return after transaction costs, because predictive accuracy and trading profitability are not equivalent criteria (Moody and Saffell 1998). The separate optimization of prediction and action also discards information: a scalar return forecast does not carry the uncertainty, timing, or context available in the original feature vector.

3.0.2. Unsupervised Learning

Unsupervised learning identifies structure in unlabeled data $\{X_i\}_{i=1}^n$ without a target variable. In trading research it is primarily used for market regime detection (clustering volatility or microstructure states), dimensionality reduction of large feature sets (PCA, autoencoders), and anomaly detection in order flow data.

These applications are useful as preprocessing or exploratory tools, but unsupervised learning does not produce trading decisions directly — it carries no reward signal and cannot optimize for a financial objective such as risk-adjusted return. In the present thesis, the regime-conditioning problem is addressed implicitly through engineered microstructure features — spread ratio, inter-event time, temporal encodings — that condition the agent's observation on the current market state without requiring an explicit unsupervised clustering step.

3.0.3. Semi-Supervised Learning

Semi-supervised learning combines a small labeled set with a larger unlabeled set, propagating label information through structural similarity in the data. It is useful when labeling is expensive or subjective — medical annotation being the canonical application.

In financial trading, the approach is rarely adopted because the main bottleneck is not label scarcity but the absence of a stable labeling criterion: what constitutes an optimal action at a given market state depends on subsequent price evolution, which is noisy and non-stationary. Constructing pseudo-labels for unlabeled market observations requires either a strong prior on optimal behavior or a separate forecasting model — at which point the problem has already been reformulated as supervised or reinforcement learning. For these reasons, semi-supervised learning is not used in this thesis.

3.0.4. Reinforcement Learning

Among the machine learning paradigms discussed above, reinforcement learning is the most directly aligned with trading as a sequential decision-making problem: an agent observes the market state, takes an action (e.g., adjusts exposure), and receives a reward that may reflect both return and risk. In contrast to supervised learning (prediction from labeled examples) and unsupervised learning (structure discovery), reinforcement learning optimizes behavior through interaction under delayed rewards and uncertainty. For this reason, the

thesis uses reinforcement learning as the core modelling framework, while the formal theoretical foundations (system components, value functions, exploration, and algorithm categories) are developed in detail in Chapter 4.

3.1. Algorithmic Trading Systems

Algorithmic trading systems can include many layers, from market-data ingestion to order routing and post-trade analysis. For the purposes of this thesis, however, only a subset of these components is directly relevant. The research question concerns whether a reinforcement learning agent can learn a useful trading policy from high-frequency order-book-derived data under a controlled simulation setup. Consequently, this section focuses on the components that are represented in the empirical design and states clearly which parts of real-world trading infrastructure remain outside the scope of the study.

3.1.1. Components Relevant to This Thesis

The empirical workflow in this thesis can be described through three functional layers:

1. a market-data and feature-construction layer,
2. a decision layer that maps states to portfolio-exposure actions,
3. an execution and portfolio-accounting layer implemented through simplified environment assumptions.

This decomposition is sufficient for analysing the learning problem addressed in the thesis without expanding the scope to a full production trading stack.

3.1.1.1. Market Data and Feature Construction

The first component is the data layer. In this thesis, the agent observes states derived from high-frequency AAPL order-book data rather than from low-frequency OHLCV bars alone. Raw order-book information is transformed into features intended to preserve short-horizon market microstructure information while remaining suitable for neural-network-based learning.

The data layer is responsible for:

- loading and validating the market dataset,
- preserving chronological ordering,

- constructing derived features from order-book information,
- separating learning features from price inputs used for portfolio valuation,
- creating train, validation, and test splits.

This layer is methodologically important because reinforcement learning in financial settings is highly sensitive to leakage, timestamp errors, and inconsistencies between engineered features and the price series used by the simulation environment.

3.1.1.2. Decision Layer

The second component is the decision layer, which is the central object of study in this thesis. The reinforcement learning agent maps the current state representation to a bounded continuous action interpreted as portfolio exposure. This means the agent learns not only directional preference, but also the magnitude of the position.

This setup differs from forecast-based trading architectures. In supervised pipelines, one model first predicts returns or labels and a separate optimization step translates those predictions into positions. In the present thesis, reinforcement learning is used as a direct decision mechanism: the policy maps observed market state directly to action, and the quality of the decision is evaluated through the reward process generated by the environment.

The main advantage of this formulation is that action selection, turnover, and reward trade-offs can be learned jointly rather than optimized in isolated stages. At the same time, it places greater importance on correct environment design, because the learned policy depends directly on how state transitions, transaction costs, and portfolio updates are represented.

3.1.1.3. Execution and Portfolio Accounting Layer

The third component is the execution and accounting layer. In real trading systems this layer may contain order management, venue selection, smart order routing, execution scheduling, fill handling, and transaction-cost analysis. These components are important in production settings, but they are not modeled in full detail in this thesis because doing so would shift the work away from the main research question and toward execution engineering.

Instead, the empirical environment uses a simplified execution abstraction. The agent's chosen action is translated into a portfolio update under explicit assumptions about pricing, transaction costs, and liquidity. This abstraction allows the study to isolate the learning

behavior of the policy while keeping the simulation tractable and reproducible.

The execution layer therefore should be interpreted as a controlled backtesting mechanism rather than a realistic implementation of exchange interaction. This distinction matters for interpretation: if the agent performs well in the simulated setting, the result supports a claim about policy learning under the specified assumptions, not a claim that the strategy is immediately deployable without further execution modeling.

3.1.2. Scope Boundary

The thesis does not attempt to model the full infrastructure of an operational algorithmic trading platform. In particular, it does not study:

- smart order routing across venues,
- partial fills and queue-position effects,
- execution algorithms such as VWAP or TWAP,
- detailed market-impact modeling,
- production monitoring, compliance, or brokerage integration.

These omitted elements are not irrelevant in practice. They are excluded because the contribution of the thesis lies elsewhere: in evaluating whether a reinforcement learning policy, trained on high-frequency order-book-derived features, can generate economically meaningful behavior within a controlled single-asset trading environment.

This scope restriction also clarifies the role of the empirical results. The reported findings should be read as evidence about the viability of the policy learning framework and the informativeness of the chosen state representation, not as a full assessment of production-grade trading-system performance.

3.1.3. Rule-Based Trading Strategies

Rule-based strategies encode trading decisions as explicit conditional logic: position changes are triggered when observable market quantities cross pre-defined thresholds. Common examples include moving-average crossover systems, momentum breakout rules, and mean-reversion strategies based on Bollinger Bands or RSI. In high-frequency settings, rule-based approaches include market-making strategies that post limit orders at fixed offsets from the mid-price and cancel them when inventory limits are reached.

The appeal of rule-based systems is transparency: the causal chain from market condition to action is explicit, auditable, and reproducible. They also impose no distributional assumptions on the data and can be deployed without a training phase.

The limitation is inflexibility. Rules are defined over a fixed set of observable conditions and optimized for a particular market regime. When the regime changes — for example, when a previously mean-reverting asset enters a trending period — fixed rules either require manual recalibration or continue generating losses until the regime reverts. This brittleness under non-stationarity is the primary motivation for learning-based approaches, which can update their behavior from experience rather than requiring explicit rule revision.

3.1.4. Trading based on Forecasts

Forecast-based trading systems employ supervised learning techniques to generate price predictions from a set of input variables. These systems optimize for statistical accuracy metrics such as mean squared error during the training phase.

The forecasting process represents merely an intermediate step in the trading workflow. The predicted prices generated by the Forecasting System subsequently inform trading decisions within the Trading Rules module. Performance evaluation occurs through financial metrics in the Profits/Losses evaluation module $U(\theta, \theta^{'})$. A significant limitation of this approach is that the Forecasting module typically outputs only the predicted price while discarding potentially valuable contextual information from the original inputs. This exemplifies a fundamental limitation in forecast-based trading systems: the information loss problem. When a system predicts that MSFT will reach $300 tomorrow based solely on technical indicators, it discards critical contextual variables such as abnormal trading volume and volatility patterns. The formal representation of this problem can be expressed as:

$$f : X \to \hat{y}$$

where: - $X \in \mathbb{R}^n$ represents the input space - $\hat{y} \in \mathbb{R}$ represents the scalar price prediction

The dimensionality reduction from $\mathbb{R}^n$ to $\mathbb{R}$ results in significant information loss that may be crucial for optimal trading decisions.

3.1.5. Training a Trading System on Labeled Data

A related approach trains a model to imitate a corpus of labeled trades directly, mapping observed market states to actions without an intermediate forecasting step. Formally, the model learns $f : X \to a$ by minimizing a loss over labeled state-action pairs (X_i, a_i^*):

$$\min_{\theta} \sum_{i=1}^{N} \ell(f_\theta(X_i), a_i^*)$$

The limitation is the same as in forecast-based approaches, expressed differently: the optimization target is the labeled action, not the trading objective. Since labels are typically constructed from historical rules or expert behavior rather than from the actual utility function, minimizing imitation error does not guarantee maximizing risk-adjusted return. The labeling process itself introduces a prior about what constitutes optimal behavior — a prior that may be wrong or regime-dependent.

3.1.6. Direct Optimization of Performance

Direct optimization removes both the intermediate forecast and the labeled action requirement. The agent observes the market state X_t, selects an action, and receives a scalar reward reflecting trading performance. It then adjusts its behavior based on that reward signal — no labels, no predictions, no proxy objective.

For example, a reinforcement learning agent trading AAPL observes the current order book state X_t and outputs a continuous portfolio exposure $a_t \in [-1, 1]$, where -1 is maximum short and $+1$ is maximum long. The reward reflects the realized return on that position net of transaction costs:

$$r_t = a_{t-1} \cdot \frac{P_t - P_{t-1}}{P_{t-1}} - c \cdot |a_t - a_{t-1}|$$

The agent optimizes a deterministic policy $\mu_\theta : X \to a$ to maximize expected cumulative reward:

$$\max_{\theta} \mathbb{E}\left[\sum_{t=1}^{T} \gamma^{t-1} r_t\right]$$

where $\gamma \in [0, 1]$ is a discount factor. The critical difference from the two preceding approaches is that the optimization objective is aligned directly with trading performance: there is no surrogate loss whose relationship to actual returns must be assumed. This alignment is the primary motivation for using reinforcement learning in this thesis.

4. Reinforcement Learning

This chapter develops the reinforcement learning foundations required for the trading agent implemented in this thesis. It is organized in three parts. The first part introduces the core components of a reinforcement learning system — the formal vocabulary of states, actions, rewards, policies, and value functions that all subsequent algorithms inherit. The second part traces the algorithmic lineage from early value-based methods to TD3, the algorithm used in the empirical work. The third part classifies RL algorithms along the dimensions that matter most for the trading problem: model-free vs. model-based, and on-policy vs. off-policy.

4.1. Components of a Reinforcement Learning System

Reinforcement learning systems address sequential decision-making problems by selecting actions that maximize cumulative discounted future rewards. This subsection defines the core components using trading as the primary illustrative context, drawing on Sutton and Barto (2018).

4.1.1. Environment

The environment defines the space of possible states and the rules governing transitions between them. In the trading setting studied in this thesis, the environment is the simulated order-book market: it receives a portfolio-exposure action from the agent at each time step, updates the portfolio value according to price changes and transaction costs, and returns the new state and reward. The environment encapsulates everything outside the agent's direct control — price dynamics, spread, and market impact — under a set of explicit simulation assumptions.

4.1.2. State

The state s_t is a snapshot of the environment at time t that contains sufficient information for the agent to select an action. In the HFT setting of this thesis, the state is a vector of engineered microstructure features derived from the limit order book: imbalance measures, spread, order flow signals, and temporal encodings. States can be terminal, ending an episode, or non-terminal, continuing the interaction sequence. The Markov property requires that the optimal action depend only on s_t and not on the full history; the feature engineering in Chapter 5 is designed to approximate this property.

4.1.3. Action

The action a_t is the decision the agent takes given state s_t. In this thesis the action space is continuous: $a_t \in [-1, 1]$, representing the target portfolio exposure from maximum short to maximum long. The continuous formulation is important because it allows the agent to express varying degrees of conviction through position size, rather than being restricted to a binary buy/sell decision.

4.1.4. Policy

The policy $\mu_\theta : \mathcal{S} \to \mathcal{A}$ maps states to actions. In this thesis a deterministic policy is used — given the same state, the policy always produces the same action. This is appropriate for a trading agent expected to behave consistently under identical market conditions. During training, exploration noise is added to the policy output to encourage discovery of profitable behaviors; at evaluation time the policy is used without noise.

4.1.5. Reward

The reward r_t is a scalar signal returned by the environment after the agent takes action a_t in state s_t. It provides the only learning signal available to the agent — there are no labels, no supervisor. The reward function is therefore a critical design choice: it determines what the agent actually optimizes. In trading, rewards are typically derived from portfolio returns, possibly adjusted for risk and transaction costs. The specific reward formulation used in this thesis is developed in Chapter 5.

4.1.6. Value Function

The value function $V^\pi(s)$ estimates the expected cumulative discounted reward starting from state s under policy π:

$$V^\pi(s) = \mathbb{E}_\pi \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \,\middle|\, s_t = s \right]$$

where $\gamma \in [0, 1]$ is the discount factor. The value function allows the agent to reason about long-run consequences rather than only immediate rewards — a necessary capability in trading, where the cost of a position change today affects future returns through transaction

costs and portfolio path dependence. The action-value function $Q^\pi(s, a)$ extends this to state-action pairs and forms the basis of the critic networks in actor-critic algorithms.

4.1.7. Applicability to Trading

Financial markets present three challenges for standard RL theory. First, transition dynamics are unknown and non-stationary — there is no closed-form model of how order book state evolves. Second, the state space is high-dimensional and continuous, making tabular methods impractical. Third, rewards are noisy and regime-dependent, making credit assignment difficult. These challenges motivate the use of model-free, off-policy, deep RL methods, and they inform the algorithm selection and evaluation design throughout this thesis.

4.2. From Tabular Methods to TD3: An Algorithmic Lineage

The development of reinforcement learning algorithms is not a linear progression of incremental improvements but a series of conceptual breaks, each motivated by a limitation in the preceding framework. This section traces the specific lineage that leads to the Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm, which is employed in this thesis. The survey is not exhaustive; it focuses on the algorithmic properties that are directly relevant to the continuous-action, single-asset trading problem formulated in Chapter 5.

4.2.1. Value-Based Methods and Their Limitations in Continuous Control

The foundational algorithms of the field — Q-learning (Watkins and Dayan 1992) and its on-policy variant SARSA (Rummery and Niranjan 1994) — operate over tabular representations of state-action value functions. Both methods converge to the optimal policy under mild conditions, but only when the state-action space is small enough to enumerate explicitly. In a trading context with continuous price-derived features, the state space is effectively infinite, and a tabular representation is computationally infeasible.

Deep Q-Networks (DQN) (Mnih et al. 2015) resolved the scalability problem by replacing the lookup table with a deep neural network, enabling generalization across states. Two architectural innovations were critical: experience replay, which decorrelates training samples by storing and randomly resampling past transitions, and a periodically frozen target network, which stabilizes the Bellman update by preventing the target from shifting at every gradient step. DQN achieved human-level control across a wide range of tasks, but it inherits a

structural constraint from Q-learning: it requires a discrete action space, because the max operator in the Bellman target presupposes that all actions can be enumerated. For a trading agent that outputs a continuous portfolio weight in $[-1, 1]$, this is not an acceptable constraint. Discretizing the action space into a finite grid resolves the representation issue formally, but it reintroduces the granularity problem: a coarse grid loses the benefits of continuous control, while a fine grid makes the maximization step computationally expensive and recovers the curse of dimensionality in higher-dimensional portfolio settings.

4.2.2. Policy Gradient Methods and the Actor-Critic Architecture

A distinct class of algorithms avoids the maximization problem by parameterizing the policy directly and optimizing it through gradient ascent on expected cumulative reward. The foundational result, REINFORCE (Williams 1992), uses Monte Carlo estimates of the policy gradient:

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} \left[\nabla_\theta \log \pi_\theta(a_t|s_t) \cdot G_t \right]$$

where G_t is the return from time t. While REINFORCE applies naturally to continuous action spaces through a parameterized distribution (for example, a Gaussian policy), it suffers from high variance in the gradient estimate because G_t is computed from a full episode trajectory, making it sensitive to trajectory length and reward scale.

The actor-critic architecture reduces this variance by replacing G_t with a learned advantage estimate $A(s_t, a_t) = Q(s_t, a_t) - V(s_t)$, maintained by a separate critic network. The actor updates the policy in the direction of higher-valued actions, while the critic provides a lower-variance baseline. This separation is the architectural foundation of all subsequent continuous-action algorithms, including TD3.

4.2.3. DDPG: Deterministic Policies for Continuous Control

Lillicrap et al. (2016) introduced Deep Deterministic Policy Gradient (DDPG), which extends the actor-critic framework to deterministic policies over continuous action spaces. Rather than sampling an action from a distribution, the actor outputs a single action $\mu_\theta(s)$ directly. The policy gradient simplifies to:

$$\nabla_\theta J(\theta) = \mathbb{E}_s \left[\nabla_a Q_\phi(s,a)\big|_{a=\mu_\theta(s)} \cdot \nabla_\theta \mu_\theta(s) \right]$$

DDPG inherits the experience replay and target network mechanisms from DQN, making it an off-policy algorithm and therefore sample-efficient. However, empirical studies consistently report two failure modes in DDPG. First, the critic tends to overestimate action values due to the maximization bias introduced when the target value is computed using the same network that is being updated. Overestimated Q-values propagate into the actor update, pushing the policy toward actions that exploit the critic's errors rather than the true reward signal. Second, the algorithm is sensitive to the relative frequency of actor and critic updates: updating the actor before the critic has converged amplifies the overestimation problem. In financial environments, where rewards are noisy and non-stationary, these instabilities are particularly damaging because the critic cannot distinguish between genuine market signal and approximation artifacts.

4.2.4. TD3: Addressing the Instability of Deterministic Actor-Critic Methods

Fujimoto, Hoof, and Meger (2018) introduced TD3 as a direct response to DDPG's failure modes, proposing three targeted mechanisms.

Clipped double Q-learning. Two independent critic networks Q_{ϕ_1} and Q_{ϕ_2} are maintained, and the Bellman target is constructed using the minimum of their estimates:

$$y = r + \gamma \min_{i=1,2} Q_{\phi_i}(s', \tilde{a}')$$

Taking the minimum provides a conservative estimate of target value and substantially reduces overestimation bias. This is analogous to the double Q-learning correction (Hasselt, Guez, and Silver 2016), adapted to the deterministic actor- critic setting.

Delayed policy updates. The actor and target networks are updated less frequently than the critics — typically once for every two critic updates. This allows the value estimates to stabilize before they are used to compute the policy gradient, breaking the feedback loop between an unstable critic and an erroneous actor update.

Target policy smoothing. When computing Bellman targets, noise is added to the target

action $\tilde{a}' = \mu_{\theta'}(s') + \epsilon$, with ϵ clipped to a small interval. This prevents the critics from fitting sharp value peaks caused by function approximation, acting as a regularizer on the value landscape.

Together, these three modifications produce an algorithm that is substantially more stable than DDPG on standard continuous control benchmarks while retaining its off-policy sample efficiency. The combination of stability under noisy rewards, data efficiency through experience replay, and native support for continuous actions makes TD3 a natural fit for the continuous portfolio allocation problem studied in this thesis. Alternative algorithms such as SAC (Haarnoja et al. 2018) incorporate maximum-entropy objectives and stochastic policies, which may improve exploration but introduce stochasticity into the policy at evaluation time — a less natural fit for a trading agent that is expected to produce consistent, interpretable allocation decisions given identical market conditions.

4.3. Classifying RL Algorithms

4.3.1. Model-free vs Model-based

Reinforcement learning algorithms divide into two broad families depending on whether they require a model of the environment.

Model-based methods learn or are given a transition model $P(s'|s, a)$ and use it to plan ahead — computing value functions by rolling out simulated trajectories rather than purely from observed data. Dyna-Q and AlphaZero are canonical examples. When the model is accurate, planning is data-efficient; when the model is wrong, planning amplifies errors. In financial markets, transition dynamics are unknown and non-stationary, making a reliable model practically unattainable.

Model-free methods learn directly from sampled transitions without building a transition model. They are less sample-efficient but impose no assumptions on the environment's dynamics. TD3 belongs to this family.

4.3.2. Value-Based, Policy-Based, and Actor-Critic

A second dimension classifies algorithms by what they learn and optimize.

Value-based methods learn a value function and derive a policy implicitly by acting greedily with respect to it. Q-learning and its deep variant DQN (Mnih et al. (2015)) are prototypical

examples. They work well in discrete action spaces but struggle when actions are continuous, because the greedy step $\arg\max_a Q(s,a)$ becomes intractable over an uncountable set.

Policy-based methods directly parameterize and optimize the policy π_θ. REINFORCE (Williams (1992)) updates parameters by gradient ascent on expected return:

$$\theta \leftarrow \theta + \alpha \nabla_\theta \mathbb{E}\left[\sum_t r_t\right]$$

Policy gradients handle continuous actions naturally but suffer from high variance, requiring many samples to obtain a reliable gradient signal.

Actor-critic methods combine both: an actor μ_θ selects actions and a critic $Q_w(s,a)$ evaluates them, using the critic's lower-variance estimates to guide actor updates. The full actor-critic derivation and TD3's specific extensions are given in Section 4.2.

4.3.3. On-Policy vs Off-Policy

On-policy methods evaluate and improve the same policy used to collect experience. SARSA is a canonical example, updating with:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)]$$

where a_{t+1} is drawn from the current policy. On-policy methods tend to be stable but wasteful: each sample is used once and then discarded.

Off-policy methods decouple the data-collection policy (the behavior policy) from the policy being improved (the target policy). Q-learning updates with the greedy maximum:

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha[r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t)]$$

regardless of how a_{t+1} was actually selected. This separation allows experience from a replay buffer — collected under earlier or exploratory policies — to be reused many times. TD3 is off-policy and uses a replay buffer to reuse collected transitions; the implications for sample efficiency in financial settings are discussed in Section 4.2.

5. Design of the Trading Agent

5.1. Action Space

The action space defines the set of permissible decisions available to the trading agent at each time step. This design choice fundamentally shapes the agent's behavior and determines the granularity of control over portfolio positions.

5.1.1. Continuous vs Discrete Actions

Trading systems can be formulated with either discrete or continuous action spaces:

Discrete action spaces restrict the agent to a finite set of categorical decisions. Common formulations include:

- Three actions: $\{-1, 0, 1\}$ representing short, neutral, and long positions
- Two actions: $\{0, 1\}$ representing short and long positions only
- Five actions: $\{-1, -0.5, 0, 0.5, 1\}$ for finer-grained position sizing

Continuous action spaces allow the agent to output any value within a specified range, enabling smooth portfolio allocation decisions.

5.1.2. Implementation: Box Portfolio Action Space

In this implementation, the action space is defined as a continuous interval:

$$\mathcal{A} = [-1.0, 1.0]$$

where the action $a_t \in \mathcal{A}$ at time t represents the target portfolio allocation for the traded asset. The semantic interpretation is:

- $a_t = 1.0$: Maximum long exposure (fully invested long position)
- $a_t = 0.0$: Neutral position (no exposure, cash equivalent)
- $a_t = -1.0$: Maximum short exposure (fully invested short position)
- $a_t \in (0, 1)$: Partial long position scaled by magnitude
- $a_t \in (-1, 0)$: Partial short position scaled by magnitude

This formulation explicitly supports short-selling, which is realistic in foreign exchange and cryptocurrency markets where symmetric long/short positions are standard. The continuous

nature enables the agent to express varying degrees of conviction—for example, $a_t = 0.3$ indicates a moderately bullish stance, while $a_t = -0.8$ signals strong bearish conviction.

5.1.3. Action Constraints and Transaction Costs

The action space is bounded to prevent unrealistic leverage. In practice, unbounded actions could lead to positions exceeding available capital or margin requirements. The $[-1, 1]$ range implicitly assumes unit leverage without margin multipliers.

Transaction costs are applied proportionally to position changes. If the agent transitions from a_{t-1} to a_t, the turnover magnitude is $|a_t - a_{t-1}|$, and fees are calculated as:

$$\text{fee}_t = \text{trading_fees} \times |a_t - a_{t-1}| \times \text{notional value}$$

This incentivizes the agent to minimize unnecessary portfolio rebalancing, as frequent adjustments erode returns. During training, the reinforcement learning algorithm must learn to balance the potential gains from reallocation against the friction costs incurred.

5.1.4. Action Smoothing and Exploration Noise

During training, exploration is introduced through additive Gaussian noise:

$$a_t^{\text{noisy}} = \text{clip}(a_t + \epsilon, -1.0, 1.0)$$

where $\epsilon \sim \mathcal{N}(0, \sigma^2)$ and σ is the exploration noise standard deviation (typically 0.1 to 0.3). The clipping operation ensures actions remain within valid bounds. This exploration mechanism encourages the agent to discover diverse trading strategies during early training phases while gradually converging to deterministic exploitation as learning progresses.

Additionally, TD3 employs target policy smoothing during critic updates, adding noise to target actions when computing Bellman targets. This regularization technique prevents the critics from overfitting to narrow peaks in the value function caused by function approximation errors.

5.2. State Space

The state space defines the informational representation available to the agent at each decision point. At each time step t, the agent receives an observation vector $s_t \in \mathcal{S}$ constructed from the current state of the limit order book (LOB). The design of this observation is the primary engineering decision in the empirical pipeline: the agent can only exploit structure that is present in its input.

5.2.1. Design Requirements

An effective state representation for high-frequency trading must satisfy several criteria simultaneously.

Approximation of the Markov property. The TD3 algorithm assumes that s_t contains sufficient information such that the optimal action depends only on the current state, not on the full history. Raw price levels violate this assumption because their unconditional distribution is non-stationary; a price of 180 means something very different depending on recent trajectory. Engineered features — returns, imbalances, normalized spreads — are designed to be more stationary and to encode the local market state rather than the absolute price level.

Informativeness about short-term price direction. For the reward signal to be learnable, the observation must contain features that are predictive of near-term mid-price changes. The selection is therefore grounded in market microstructure theory, which identifies specific LOB quantities — imbalances, order flow, spread — as the empirically dominant predictors of short-term price movement.

Dimensionality discipline. Each additional observation dimension enlarges the input space that the policy and critic networks must generalize over. Features are included only when they carry a distinct theoretical mechanism not already captured by other features in the set.

Scope and timescale consistency. The observation is restricted to LOB-derived features. Bar-based signals such as OHLCV aggregates or technical indicators require a fixed aggregation window that imposes a coarser timescale than the tick-level decisions being made: a one-second bar collapses dozens of individual order-book events into a single price summary, discarding the queue dynamics, imbalance shifts, and spread fluctuations that constitute the decision-relevant information at this resolution. Cross-asset signals such as VIX or index

returns face a related problem: they update at frequencies far below the inter-event times of the AAPL tick stream, contributing no new information between consecutive book updates. Aligning multiple tick streams at sub-second precision also introduces synchronization risk — a VIX observation and an AAPL LOB snapshot sharing a timestamp to the second are not the same market moment. Beyond these practical constraints, restricting the state to single-asset LOB features is a deliberate experimental choice: the thesis tests whether microstructure-aware features alone are sufficient for a viable agent. Adding cross-asset signals would answer a different question and prevent clean attribution of any observed performance to LOB structure. A natural extension for future work would be to augment the LOB observation with synchronized regime indicators — intraday VIX futures at the minute level, sector ETF order flow, or options-derived skew — once the baseline LOB-only performance is established, allowing a controlled ablation of which additional signal sources materially improve out-of-sample results.

5.2.2. Feature Groups and Selection Rationale

The state vector is composed of five groups of features, each targeting a distinct aspect of LOB microstructure.

5.2.2.1. Imbalance Features

Volume imbalance at a given price level i is defined as:

$$\text{BookPressure}_i = \frac{V_i^{bid} - V_i^{ask}}{V_i^{bid} + V_i^{ask}}$$

Imbalance signals the relative weight of resting liquidity on each side and has been shown to predict short-term price direction (Cao, Chen, and Griffin 2004; Biais, Hillion, and Spatt 1995). The multi-level aggregate version averages this signal across the top N levels:

$$\text{OBI}_N = \frac{\sum_{i=0}^{N-1}(V_i^{bid} - V_i^{ask})}{\sum_{i=0}^{N-1}(V_i^{bid} + V_i^{ask})}$$

The multi-level formulation is more robust to noise at the best quote and captures the depth structure of the book. Both single-level (levels 0–2) and three-level aggregate variants are included.

Order count imbalance complements the volume-based measures by counting the number of resting orders rather than their aggregate size, making it sensitive to fragmentation that a single large institutional order would mask:

$$\mathrm{OCI}_i = \frac{C_i^{bid} - C_i^{ask}}{C_i^{bid} + C_i^{ask}}$$

5.2.2.2. Fair Value Features

The microprice (Stoikov 2018) is a volume-weighted estimate of where the next transaction price is likely to occur:

$$\text{Microprice} = \frac{V_0^{ask} \cdot P_0^{bid} + V_0^{bid} \cdot P_0^{ask}}{V_0^{bid} + V_0^{ask}}$$

When the ask side is thin relative to the bid side, the microprice is pulled toward the ask — the side more likely to move first. The microprice divergence from mid-price isolates the imbalance component of fair value as a stationary signal:

$$\text{MicroDiv} = \text{Microprice} - \frac{P_0^{bid} + P_0^{ask}}{2}$$

The volume-weighted mid-price skew extends this concept across three levels, measuring whether deep liquidity is positioned asymmetrically relative to the current mid-price.

5.2.2.3. Spread and Cost Features

The bid-ask spread, expressed in basis points, is the primary observable measure of adverse selection cost (Kyle 1985; Glosten and Milgrom 1985):

$$\text{SpreadBps} = \frac{P_0^{ask} - P_0^{bid}}{\frac{P_0^{bid} + P_0^{ask}}{2}} \times 10{,}000$$

A widening spread signals elevated information asymmetry or inventory risk among market makers. The spread ratio — current spread relative to its recent rolling mean — separates the level effect from the regime signal, flagging periods of abnormally elevated adverse selection risk (Easley, López de Prado, and O'Hara 2012).

Bid and ask slopes measure the price elasticity of each side of the book across five levels (Bouchaud, Mézard, and Potters 2002), capturing market impact per unit of trading volume and serving as a proxy for Kyle's lambda (Kyle 1985):

$$\text{BidSlope} = \frac{P_0^{bid} - P_4^{bid}}{\sum_{i=0}^{4} V_i^{bid}}, \qquad \text{AskSlope} = \frac{P_4^{ask} - P_0^{ask}}{\sum_{i=0}^{4} V_i^{ask}}$$

5.2.2.4. Order Flow Features

Order flow imbalance (OFI) (Cont, Kukanov, and Stoikov 2014) measures the net directional pressure from LOB events between consecutive snapshots, conditioning on whether each event represents genuine queue growth rather than a price improvement:

$$\text{OFI}_t = \Delta V_t^{bid} \cdot \mathbf{1}\left[P_{0,t}^{bid} \geq P_{0,t-1}^{bid}\right] - \Delta V_t^{ask} \cdot \mathbf{1}\left[P_{0,t}^{ask} \leq P_{0,t-1}^{ask}\right]$$

Cont, Kukanov, and Stoikov (2014) establish OFI as the single strongest predictor of short-term price changes in LOB data. A rolling sum over 50 events captures persistent directional pressure over multiple tick horizons.

Signed trade flow (cumulative delta) (C. M. C. Lee and Ready 1991) is constructed directly from the trade tape rather than from book changes, providing a complementary signal that measures executed volume rather than queue dynamics:

$$\text{CumDelta}_{W,t} = \sum_{\tau=t-W+1}^{t} \text{size}_\tau \cdot \mathbf{1}[\text{side}_\tau = B] - \text{size}_\tau \cdot \mathbf{1}[\text{side}_\tau = A]$$

Queue depletion measures how rapidly the best-bid or best-ask queue is being consumed across consecutive snapshots, identifying situations where the top-of-book queue is close to exhaustion and a price move is imminent.

Odd-lot trade ratio and odd-lot imbalance (Boehmer et al. 2021) decompose trade flow by order size. In the AAPL dataset used here, 81% of trades are odd-lot sized (below 100 shares), primarily reflecting retail and algorithmic small-lot activity. A sudden drop in the odd-lot ratio signals increasing institutional participation and consequently higher adverse selection risk.

5.2.2.5. Regime and Temporal Features

Two features condition the agent on the current market regime rather than providing directional signals.

Log inter-event time (Engle 2000) captures the activity rate of the market:

$$\mathrm{LogGap}_t = \log\big(1 + (t_{\mathrm{ns}} - t_{\mathrm{ns},t-1})\big)$$

Dense update bursts indicate informed order flow or a large order working through the book; sparse updates signal low activity and reduced adverse selection risk.

Mid-price acceleration is the second finite difference of the mid-price:

$$\mathrm{Accel}_t = M_t - 2M_{t-1} + M_{t-2}, \qquad M_t = \frac{P_0^{bid} + P_0^{ask}}{2}$$

It distinguishes momentum regimes (positive, accelerating) from mean-reversion regimes (negative, decelerating) at tick frequency.

The sine and cosine encodings of the hour-of-day capture the well-documented U-shaped intraday pattern in volume and spread (Wood, McInish, and Ord 1985):

$$\mathrm{HourSin}_t = \sin\left(\frac{2\pi \cdot h_t}{24}\right), \qquad \mathrm{HourCos}_t = \cos\left(\frac{2\pi \cdot h_t}{24}\right)$$

The cyclical encoding ensures temporal continuity — 09:30 and 16:00 remain structurally distinct without imposing an arbitrary ordinal distance.

5.2.2.6. Position Feature

The observation vector is optionally extended with the agent's own current portfolio exposure $a_{t-1} \in [-1, 1]$. Unlike the LOB-derived features, this quantity is not computed from market data but read from the simulated broker state at each step — it is a runtime feature appended after the static feature vector is constructed.

Including the current position makes the state partially Markovian with respect to inventory: without it, the agent cannot distinguish between arriving at exposure 0 from a long position

versus arriving from a short position, even though the cost of the next action differs in both cases through the position-change fee. With it, the agent can condition its action on whether a change would incur a fee and can learn to penalize unnecessary turnover. The feature requires no normalization as it is already bounded in $[-1, 1]$ by the action space definition.

5.2.3. Observation Space and Normalization

The complete observation vector $s_t \in \mathbb{R}^d$ is formed by concatenating all engineered features. The exact dimensionality d is scenario-dependent and specified in the configuration files governing each experiment.

All features except cyclical time encodings are standardized using training-set statistics:

$$\tilde{x}_i = \frac{x_i - \mu_i^{\text{train}}}{\sigma_i^{\text{train}}}$$

The mean μ_i^{train} and standard deviation σ_i^{train} are computed exclusively on the training split and applied without modification to the validation and test splits, preventing any form of data leakage. Cyclical encodings are inherently bounded in $[-1, 1]$ and require no normalization.

The formal observation space is:

$$\mathcal{S} = \mathbb{R}^d$$

In practice, after standardization, feature values concentrate in $[-3, 3]$ under approximate Gaussian assumptions. The policy and value networks must generalize to out-of-distribution observations that arise during stress episodes or regime shifts not present in the training sample.

5.2.4. On the Exclusion of Raw LOB Columns

Raw price and size columns — that is, the unprocessed bid and ask prices and sizes at each level — were deliberately excluded from the observation vector. Prior work in the deep learning tradition feeds the raw LOB matrix directly to the network (Z. Zhang, Zohren, and Roberts 2019), allowing convolutional or recurrent architectures to discover structure from data without hand-engineered intermediaries. This is a coherent approach when the model

itself is the feature extractor.

In the present setup, however, comprehensive microstructure features with established theoretical mechanisms are the primary input to the agent. Including raw columns alongside them introduces redundancy without adding an interpretable signal: normalized bid sizes at each level carry substantially the same information as the imbalance features derived from them, and normalized absolute prices at tick frequency are near-meaningless once the local market state is already encoded through microprice divergence, OFI, and spread measures. Mixing both paradigms would imply that the engineered features are insufficient, which is inconsistent with the selection rationale described above. The observation vector is therefore composed entirely of theoretically motivated derived features.

5.3. Reward Function

The reward function determines what the agent actually optimizes. In trading, two properties are essential: the signal should be risk-adjusted so the agent does not maximize returns by taking excessive volatility, and it should be computable recursively so that no look-ahead into future returns is required at each training step.

5.3.1. Why Raw Log-Return Is Insufficient

The simplest reward is the single-step log-return:

$$r_t = \log \frac{P_t}{P_{t-1}} \cdot a_{t-1}$$

At tick frequency, this signal is dominated by microstructure noise. Individual price changes on an MBP-10 order book are a fraction of the spread; the agent cannot distinguish genuine directional movement from quote flickering in a single step. Training on raw log-return produces highly variable gradient estimates and tends to reward high-turnover behavior regardless of risk.

A related alternative — the Sharpe ratio computed over a fixed rolling window — requires storing the full window of past returns at each step and introduces a sharp discontinuity when old returns drop out of the window. Neither property is desirable in an online RL setting.

5.3.2. Differential Sharpe Ratio

The Differential Sharpe Ratio (Moody and Saffell 1998) resolves both issues. It is defined as the first-order approximation of how the current return R_t changes the exponentially weighted Sharpe ratio:

$$D_t = \frac{B_{t-1}\,\Delta A_t - \frac{1}{2}A_{t-1}\,\Delta B_t}{\left(B_{t-1} - A_{t-1}^2\right)^{3/2}}$$

where A_t and B_t are exponentially weighted estimates of the first and second moments of returns:

$$A_t = \eta R_t + (1-\eta)\,A_{t-1}, \qquad B_t = \eta R_t^2 + (1-\eta)\,B_{t-1}$$

and $\Delta A_t = A_t - A_{t-1}$, $\Delta B_t = B_t - B_{t-1}$. At $t = 0$, $A_0 = B_0 = 0$.

The smoothing parameter η controls the effective memory of the estimator: larger η makes the estimate more reactive to recent returns, smaller η averages over a longer history. The effective window length is approximately $1/\eta$ steps. In the empirical experiments, $\eta = 0.01$, giving an effective memory of roughly 100 ticks — long enough to smooth microstructure noise while short enough to adapt to intraday regime shifts.

The DSR has two properties that make it well-suited to this setting. First, it penalizes return volatility intrinsically: a sequence of returns with the same mean but higher variance produces a lower DSR, which discourages the agent from achieving returns through high-variance position flipping. Second, it is recursively computable from A_{t-1} and B_{t-1} alone — no fixed window, no look-ahead, and no discontinuity as old observations expire.

5.3.3. Reward and Financial Return Are Not the Same

A shaped reward such as DSR is used to stabilize learning; it is not equivalent to the financial performance metrics reported in Chapter 7. The Sharpe ratio, annualized return, and drawdown statistics reported in the evaluation chapter are computed from the realized log-return series of the deterministic policy rollout, not from the cumulative DSR accumulated during training. Conflating the two — for example, treating a high training DSR as evidence of a strong Sharpe ratio on the test set — would invalidate the empirical comparison.

5.4. Value Function

Chapter 4 introduced the state-value function $V^\pi(s)$ and action-value function $Q^\pi(s,a)$ as the expected cumulative discounted return under policy π. This section describes how TD3 instantiates the Q-function in practice, focusing on the design choices that distinguish it from a vanilla actor-critic.

5.4.1. Twin Critics

TD3 maintains two independent Q-networks Q_{θ_1} and Q_{θ_2}. The Bellman target is computed using the minimum of the two estimates:

$$y_t = r_t + \gamma \min_{i=1,2} Q_{\theta'_i}(s_{t+1}, \tilde{a}_{t+1})$$

where θ'_i are slowly updated target network parameters and $\tilde{a}_{t+1} = \mu_{\phi'}(s_{t+1}) + \epsilon$ is the target action with smoothing noise. Taking the minimum counteracts the overestimation bias that arises when bootstrapped targets are computed from the same network being updated — a failure mode documented in DDPG and addressed here following Fujimoto, Hoof, and Meger (2018).

Both critics are trained by minimizing the TD error against the shared target y_t. The actor is updated using only the first critic's gradient:

$$\nabla_\phi J(\phi) = \mathbb{E}_{s\sim\mathcal{D}}\left[\nabla_a Q_{\theta_1}(s,a)\big|_{a=\mu_\phi(s)} \nabla_\phi \mu_\phi(s)\right]$$

The second critic contributes only through the conservative Bellman target, not directly to the policy gradient.

5.4.2. Loss Function

The critics are trained with Smooth L1 (Huber) loss rather than MSE:

$$\mathcal{L}(\delta) = \begin{cases} 0.5\,\delta^2 & |\delta| \le 1 \\ |\delta| - 0.5 & \text{otherwise} \end{cases}$$

where $\delta = y_t - Q_\theta(s_t, a_t)$ is the TD error. The linear tail limits the influence of large TD errors that occur during volatile market episodes, providing robustness that MSE does not.

5.5. Policy

5.6. Discount Factor

The discount factor $\gamma = 0.99$ is used throughout this thesis. The role of γ in the value function is defined in Chapter 4; this section justifies the specific value chosen.

With $\gamma = 0.99$ the effective planning horizon is approximately $1/(1-\gamma) = 100$ steps. On the HFT data used here — where each step corresponds to a short intraday interval — this horizon covers a meaningful window of market activity without extending so far that value estimates accumulate excessive variance from stochastic rewards.

$\gamma = 0.99$ is held constant across all experiments to isolate the effect of other design choices (state representation, reward function). Sensitivity to γ is a natural direction for future work, particularly whether lower values (0.9–0.95) improve performance in choppy, mean-reverting regimes.

5.7. Optimization Hyperparameters

5.7.1. Learning Rates

Both actor and critic use Adam (Kingma and Ba 2014) with a learning rate of $\alpha = 0.0001$. This is deliberately conservative relative to the 3×10^{-4} default common in continuous-control benchmarks. Preliminary experiments with more aggressive rates produced Q-value explosions and policy collapse — failure modes consistent with the high noise-to-signal ratio of tick-frequency rewards and the sensitivity of bootstrapped TD targets to large gradient steps. Stability was prioritized over sample efficiency.

Actor and critic use the same nominal rate, but TD3's policy delay (every $d = 2$ critic updates) effectively halves the actor's update frequency, providing additional stability without a separate hyperparameter.

5.7.2. Weight Decay

L2 weight decay of $\lambda = 2 \times 10^{-6}$ is applied to both networks. The magnitude is intentionally small — sufficient to discourage memorization of training-period noise without restricting

the network's capacity to represent nonlinear market dynamics. No further regularization (dropout, batch normalization) is used, keeping the architecture parsimonious.

5.7.3. Scope

Learning rate and weight decay values were selected through preliminary stability checks rather than systematic search. They should be interpreted as the configuration used for the reported experiments, not as globally optimal values. Sensitivity to these parameters is a natural direction for future work.

6. Implementation of the Trading Agent

This chapter describes how the design choices from Chapter 5 are realized in the experimental pipeline. It covers data preparation, code organization, and the TD3 training workflow.

6.1. Simulation Assumptions

The following assumptions govern the trading simulation throughout all experiments.

Zero market impact and no order-book interaction. Agent trades are assumed not to move quoted prices, consume displayed depth, or alter subsequent order-book states. The environment therefore treats the agent as price-taking rather than market-shaping. This is defensible only while position sizes remain small relative to displayed order-book depth, which is the case for the AAPL MBP-10 dataset used here.

Mid-price execution. Trades are valued at the mid-price rather than at executable bid/ask quotes. This avoids explicit spread-crossing and simplifies reward accounting, but it implies that reported backtest returns are an upper bound on achievable net performance. Real execution must cross the spread, and realized returns are further reduced by maker/taker fees, queue position, and partial fills.

Zero latency. Observations are assumed to be acted on immediately, with no transport, decision, or matching-engine delay between state observation and execution. This is appropriate for a study focused on strategy learning rather than low-latency execution engineering. In production HFT systems, such delays can invalidate short-lived signals.

Bounded continuous exposure. Position constraints are enforced via the continuous action interval $[-1, 1]$, preventing unbounded leverage.

Transaction fee. A configurable fee parameter is applied per position change. Baseline runs use a zero-fee setting; sensitivity experiments vary this parameter.

These assumptions collectively make the simulation tractable and the results interpretable as learning-dynamics benchmarks rather than deployment-ready performance estimates.

6.2. Data Preparation

6.2.1. Input Format

The pipeline consumes time-indexed LOB snapshot data. Each row represents an order-book event with multi-level bid/ask prices, sizes, and order counts, plus trade descriptors used for order-flow features. The core column groups are summarized below.

Table 2: Input schema used in the empirical pipeline

Column group	Description	Role
`timestamp`	Chronological event index	Ordering, temporal features
`bid_px_NN`, `ask_px_NN`	Bid/ask prices at level `NN`	Spread, mid-price, slope, valuation
`bid_sz_NN`, `ask_sz_NN`	Resting size at level `NN`	Imbalance, depth, OFI, queue features
`bid_ct_NN`, `ask_ct_NN`	Order count at level `NN`	Order count imbalance
`action`, `side`, `size`	Event type, aggressor side, size	Signed trade flow features
`close` (auxiliary)	Mid-price-like pricing series	Reward and portfolio valuation

6.2.2. Preprocessing and Splitting

Raw data is validated for chronological order, required column presence, non-negative prices, and non-crossed best quotes before any feature engineering. This stage is conservative: its purpose is correctness, not statistical filtering.

The dataset is then split chronologically into training, validation, and test subsets. Shuffling is not used because it would destroy temporal structure and introduce information from future periods into earlier training samples. The training split is used to fit model parameters; the validation split supports configuration comparison and development diagnostics; the test split is reserved for final out-of-sample reporting and does not influence any model selection decisions.

6.2.3. Feature Normalization and Leakage Prevention

Normalization statistics (mean and standard deviation for z-score scaling) are estimated on the training split only and applied unchanged to the validation and test splits. Fitting normalization on the full dataset would allow future volatility regimes to influence the training representation, inflating apparent performance. Cyclical time encodings (hour-of-day sine and cosine) are already bounded in $[-1, 1]$ and are not normalized. Raw price columns used for portfolio accounting are kept in their original units and are never replaced by normalized values.

6.2.4. Feature and Price Column Separation

A strict separation is maintained between the observation features passed to the learning algorithm and the price inputs used by the portfolio simulation. The learning features are normalized and engineered; the pricing columns are economically interpretable and used only for reward and valuation computation. Mixing these two roles can produce invalid reward calculations or cause normalized prices to appear in financial metrics.

In the current implementation, the auxiliary pricing series used for portfolio valuation plays the role of a mid-price proxy rather than an executable bid/ask quote. This simplifies reward accounting but makes the resulting backtest performance optimistic relative to realistic spread-crossing execution.

6.3. Implementation and Training Pipeline

6.3.1. Configuration-Driven Design

Each experiment is fully specified by an external YAML scenario file covering data paths, feature subsets, reward type, environment settings, and optimization hyperparameters. No source-code changes are required to switch between experimental variants. A pre-training validation stage checks path availability, feature definitions, split feasibility, and column compatibility before any computation begins, preventing wasted runs from misconfigured experiments. A condensed example configuration is provided in Appendix C.

6.3.2. Feature Pipeline

Feature engineering is registry-driven: feature names in the scenario file resolve to computation functions, allowing scenario-level composition without editing training code. This design is central to systematic feature subset comparisons. Normalization parameters are estimated on the training split only and applied without re-fitting to the validation and test data, preserving causal integrity across the chronological split.

6.3.3. Training Loop

The processed training split is used to construct the trading environment with a continuous action space $a_t \in [-1, 1]$ representing target portfolio exposure. Training follows the TD3 procedure described in Chapter 4: experience is collected into a replay buffer, twin critics are updated from sampled mini-batches, and the actor is updated every d critic steps via the deterministic policy gradient. Target networks track the main networks through soft updates with coefficient τ. Exploration noise is applied during training and removed at evaluation time.

6.3.4. Evaluation

Final performance is reported on the held-out test split using a separate evaluation environment. Two output streams are kept strictly separate: the training reward (an optimization diagnostic) and the realized portfolio return series from which financial metrics are computed. Conflating these — for example treating cumulative DSR as a Sharpe ratio estimate — would invalidate the empirical comparison. Metrics derived from the test rollout are described in Chapter 7.

7. Empirical Evaluation and Performance Analysis

This chapter reports the empirical results for the thesis-facing experiment: the TD3 agent trained on high-frequency AAPL order-book-derived features under the state-space design defined in Chapters 5 and 6. The purpose of the chapter is not merely to display training outputs, but to assess whether the configured agent produces economically meaningful behavior on held-out data once transaction costs, turnover, and run-to-run variability are taken seriously.

To keep the rendered thesis aligned with the recorded experiments, tables and figures are loaded from exported result snapshots generated by the experiment tracking workflow. When no exported snapshot is available, the chapter can fall back to the local tracking store.

Two result views are used throughout:

- a monitoring view of the latest active run, when training is still in progress
- an evaluation view of the latest finished run with test-set artifacts

7.1. Experiment Snapshot

7.2. Statistical Validation

This section interprets the empirical results of the thesis experiment through the lens of training stochasticity. For the present AAPL high-frequency TD3 setup, a single favorable run is not sufficient evidence: repeated runs can diverge because of random initialization, replay sampling, and exploration noise even when the dataset and configuration are fixed. Statistical validation is therefore used here to quantify run-to-run dispersion and to distinguish stable behavior from isolated favorable trajectories.

The analysis operates on exported thesis result snapshots so that the rendered chapter stays tied to recorded experiment artifacts.

7.2.1. Purpose and Scope of Statistical Validation

In the current work, statistical validation serves three narrower purposes. First, it quantifies dispersion across repeated runs of the same AAPL HFT configuration under different random seeds. Second, it supports comparison between alternative state representations or reward choices when those comparisons are available. Third, it prevents overinterpretation of a

single run whose performance may reflect favorable stochastic training dynamics rather than repeatable policy quality.

The present thesis distinguishes between two classes of quantities:

1. training-time monitoring metrics, which are useful for diagnosing learning dynamics but are not sufficient as final evidence of financial performance;
2. final evaluation metrics, which summarize realized portfolio behavior and are the primary basis for empirical conclusions.

This distinction is essential because low optimization loss or improving training diagnostics do not necessarily imply superior out-of-sample trading performance.

7.2.2. Run-Level Sample Summary

The following table summarizes currently available finished runs for the selected experiment. These values should be interpreted as a sample of outcomes under the configured workflow rather than as definitive performance estimates.

7.2.3. Descriptive Statistics and Uncertainty Estimates

For repeated runs of the same configuration, descriptive statistics provide the first indication of stability. In this thesis, quantities such as the sample mean, standard deviation, and range should be read as summaries of variability under the reported protocol, not as universal claims about future market performance. Repeated backtests quantify sensitivity to training randomness more than they resolve uncertainty about the underlying market process.

7.2.4. Comparative Validation Strategy for Thesis-Grade Results

When comparative experiments are reported, they should be interpreted under a matched design: the data split, cost assumptions, and evaluation procedure must remain fixed while only the feature set, reward specification, or other target variation is changed. Within that design, the preferred interpretation hierarchy is:

1. effect size (difference in mean or median performance),
2. dispersion (stability across runs),
3. confidence interval overlap or formal hypothesis testing,
4. economic plausibility (e.g., turnover, drawdown, net performance after costs).

This ordering matters because statistically detectable differences can still be economically unimportant if they coincide with unstable turnover, weak net returns after costs, or poor drawdown behavior.

7.2.5. Statistical Risks and Interpretation Caveats

Several risks remain when interpreting these summaries. Repeated runs on the same historical period are not independent samples from a stable data-generating process; they quantify training stochasticity more than market uncertainty. In addition, repeated tuning on the same validation period can bias later comparisons, and reward-based metrics can diverge from financial objectives when the reward is a shaped training signal rather than realized return.

Accordingly, statistical validation in this thesis is treated as a method for ranking the credibility of observed results under a controlled protocol, not as proof of persistent real-world profitability. Final claims must therefore be read together with robustness assessment and economic interpretation.

7.3. Robustness Assessment

Robustness assessment asks whether the conclusions drawn from the current AAPL high-frequency TD3 experiment survive changes in the assumptions most likely to matter for this thesis. Here the central question is not generic algorithm robustness in the abstract, but whether the reported behavior depends too heavily on a particular random seed, feature specification, reward definition, or transaction-cost setting.

The goal is therefore to identify which parts of the empirical result look structural and which may still be artifacts of a narrow experimental setup.

7.3.1. Why Robustness Matters in This Thesis

In this work, robustness matters because the agent is trained on a noisy microstructure-driven reward stream and evaluated in a simplified execution environment. Apparent improvements may therefore come from favorable training dynamics or feature-set idiosyncrasies rather than from a genuinely more useful policy. A result is treated as robust only if it remains directionally consistent under reasonable changes in seed, state representation, reward specification, and cost assumptions, while preserving interpretable turnover and drawdown behavior.

7.3.2. Current Evaluation Context

The following snapshot provides the current experiment context used in the automated Chapter 7 result wiring. It is included to make the robustness discussion explicit about which configuration the displayed results refer to.

7.3.3. Robustness Dimensions Considered in This Thesis

The robustness framework in this thesis is centered on five dimensions most directly connected to the current empirical design:

1. **Random-seed sensitivity**: whether performance is stable across repeated runs of the same configuration.
2. **Feature-set sensitivity**: whether the richer state-space design improves results relative to simpler snapshot-heavy alternatives without introducing unstable behavior.
3. **Reward-function sensitivity**: whether conclusions depend on the specific training objective used by the agent.
4. **Transaction-cost sensitivity**: whether apparent gains survive realistic fee assumptions rather than being generated by excessive turnover.
5. **Data-split sensitivity**: whether results remain similar under reasonable changes in the chronological split or training-window length.

These dimensions are sufficient to test whether the present thesis conclusions are specific to one narrow configuration or remain credible under nearby variations.

7.3.4. Robustness Matrix for Experimental Reporting

The table below defines a practical robustness matrix for the empirical part of the thesis. It can be used as a reporting checklist when final results are compiled.

7.3.5. Interpreting Noisy Training Curves in Robustness Terms

Noisy training curves are common in actor-critic methods and should not be treated as the primary robustness criterion. In the present workflow, the more informative question is whether the same configuration produces similar evaluation outcomes across repeated runs and whether qualitative behavior, such as persistent overtrading or directional bias, changes systematically when the feature set, reward, or cost assumptions are varied.

This is particularly relevant for the present thesis because one of the central comparisons concerns simpler versus broader order-book-derived state representations. If the richer feature set improves returns but materially increases instability or turnover, the result is weaker than a modest but more stable improvement.

7.3.6. Methodological Limits of the Current Robustness Snapshot

The current snapshot-based Chapter 7 integration supports reproducible loading of run summaries and evaluation artifacts, but robustness conclusions remain conditional on the set of experiments already executed and exported. A single finished run can illustrate workflow behavior, but it cannot support strong claims about robustness across seeds, feature sets, or cost regimes.

For this reason, robustness findings in the final thesis should be framed only to the extent supported by repeated runs and matched comparison settings. The framework in this section provides the structure for that interpretation and helps prevent post-hoc emphasis on isolated favorable outcomes.

7.4. Performance Evaluation

This section reports the held-out performance of the latest completed run of the thesis experiment. The aim is not just to list metrics, but to judge whether the TD3 agent's behavior remains economically coherent once returns, turnover, and path-dependent risk measures are inspected together. Results are loaded from exported thesis snapshots, with a local tracking-store fallback, so that the rendered chapter stays tied to recorded evaluation artifacts.

7.4.1. Selected Metrics from the Latest Completed Run

7.4.2. Evaluation Metadata Snapshot

7.4.3. Evaluation Plots

7.4.4. Discussion: Reading Performance in the Context of State Design

The key interpretive issue in this thesis is not whether the agent attains a single high metric, but whether the reported behavior is consistent with the microstructure-aware state design proposed in Chapter 5. Because the current work studies order-book-derived state representations, performance should be read jointly with turnover, drawdown, and action behavior

rather than through return metrics alone.

This matters especially when comparing simpler snapshot-heavy feature sets with broader state representations that also encode flow, timing, and spread-regime information. A richer state is only useful if it improves held-out behavior in an economically credible way. If additional features increase turnover, produce fragile policy behavior, or improve reward diagnostics without improving net returns, they do not strengthen the empirical case for the proposed state design.

For that reason, the performance tables in this section should be interpreted as evidence about the quality of the overall decision pipeline: state representation, reward specification, and policy learning together. The central question is whether the broader microstructure-aware feature design yields more stable and more interpretable out-of-sample behavior than simpler alternatives, not whether any single metric is maximized in isolation.

8. Conclusions and Future Work

8.1. Summary of Findings

8.2. Limitations and Future Research

8.2.1. Order-Book Execution Realism Constraint

A central limitation of the current empirical setup is the use of a mid-price-like execution proxy for portfolio valuation and reward computation. This assumption is convenient for controlled RL training, but it is not fully realistic for live order-book trading.

In production settings, a strategy that demands immediate execution typically crosses the spread, which introduces a mechanical cost on each trade. Realized results are also affected by maker/taker fees, queue priority, partial fills, and latency-dependent adverse selection. These frictions are only partially represented in the present setup.

Relatedly, the current environment assumes zero market impact: agent trades do not shift prices or deplete liquidity in a way that changes subsequent execution conditions. This is reasonable only for small orders relative to available depth. Relaxing this assumption would likely lower net returns, increase drawdowns, and penalize high-turnover behavior through impact-driven slippage and adverse selection.

The setup also assumes zero latency between observation and execution. This is acceptable for the present research objective because the thesis does not target latency-arbitrage deployment; however, real HFT environments are highly latency-sensitive. Some signals that appear profitable in simulation may decay before execution in live markets, making them effectively non-tradeable. Additionally, real venues can reject or partially reject orders due to insufficient margin, stale quotes, risk limits, or rapid order-book changes, which can further degrade realized performance relative to backtest results.

The practical implication is that reported backtest performance is likely optimistic relative to deployable net performance. Strategies with high turnover are especially sensitive: at tick frequency, spread-crossing costs can be of similar magnitude to the short-horizon price moves the agent attempts to capture, meaning that a strategy appearing profitable at mid-price may generate neutral or negative returns under realistic bid/ask execution. Future research should therefore evaluate the same agents under explicit bid/ask execution rules, calibrated fee schedules, and slippage/impact models to quantify the robustness of the observed edge

under realistic microstructure constraints.

8.2.2. Agent-State Feature Extensions

The current observation vector includes the agent's own portfolio exposure as the sole agent-specific feature. A natural extension is to enrich this with additional inventory-aware signals that describe the agent's situation within the episode rather than the external market state. Candidate features include unrealized P&L on the current position — which would allow the agent to distinguish a winning long from a losing long and potentially implement implicit stop-loss or profit-taking behavior — the number of steps the current position has been held, which enables time-based exit logic, and a rolling turnover count that makes the agent aware of its own recent transaction cost footprint. At a higher level, the agent's drawdown from its episode peak could support risk-sensitive behavior during losing runs without requiring explicit rule-based overlays. These extensions leave the market observation unchanged and add only internal state, making them a low-disruption way to test whether richer self-awareness improves policy quality.

8.2.3. Cross-Asset and Multi-Asset Extensions

The empirical design is restricted to a single asset — AAPL — under the assumption that the LOB microstructure of that instrument alone contains sufficient signal. A direct extension is to introduce synchronized cross-asset observations: intraday VIX futures as a regime-conditioning signal, sector ETF order flow as a proxy for institutional activity, or correlated large-cap LOB states that may lead AAPL price discovery. A more substantial extension is to move from single-asset exposure control to multi-asset portfolio optimization, where the agent simultaneously manages positions across several instruments and must account for correlation, capital constraints, and cross-asset execution costs.

8.2.4. Generalization Across Asset Classes and Market Regimes

The present experiments are conducted on equity tick data from a three-day window. A key open question is whether the learned representations and policy behavior transfer across asset classes — foreign exchange, futures, and cryptocurrency markets each exhibit distinct microstructure: different spread regimes, tick sizes, liquidity profiles, and intraday seasonality. Testing the same feature design and training protocol across these settings would establish

whether the approach is robust to structural differences in order-book dynamics, or whether asset-class-specific feature engineering and reward tuning are necessary. Extending the training window to cover multiple market regimes — including periods of elevated volatility, low liquidity, and structural breaks — would also strengthen the generalization claim beyond the narrow three-day sample used here.

8.2.5. Algorithm Comparison and Reward Ablation

The empirical pipeline is structured around TD3 and the Differential Sharpe Ratio reward. Two natural ablation axes remain unexplored. First, replacing TD3 with SAC — which maximizes a maximum-entropy objective and tends to maintain broader exploration throughout training — or with PPO would test whether the deterministic actor-critic family is genuinely best suited to this problem. Second, replacing the DSR reward with alternatives such as a Sortino-based signal, a direct log-return objective, or a reward that explicitly penalizes drawdown would test the sensitivity of policy behavior to reward specification — a dimension the thesis identifies as important but does not empirically vary.

8.2.6. Evaluation Robustness

The current results are based on a fixed train/test split and a single random seed. Walk-forward validation — expanding the training window and re-evaluating on successive out-of-sample periods — would provide a more rigorous assessment of temporal generalization. Multi-seed reporting with confidence intervals would separate genuine policy quality from favorable initialization, which is a known confound in RL research.

8.3. Implications for Trading Systems

8.4. Recommendations for Practitioners

8.5. Conclusion

List of Figures

Bibliography

Bellman, Richard. 1957. *Dynamic Programming*. Princeton, NJ: Princeton University Press.

Biais, Bruno, Pierre Hillion, and Chester Spatt. 1995. "An Empirical Analysis of the Limit Order Book and the Order Flow in the Paris Bourse." *Journal of Finance* 50 (5): 1655–89. https://doi.org/10.1111/j.1540-6261.1995.tb05192.x.

Boehmer, Ekkehart, Charles M Jones, Xiaoyan Zhang, and Xinran Zhang. 2021. "Tracking Retail Investor Activity." *Journal of Finance* 76 (5): 2249–2305. https://doi.org/10.1111/jofi.13033.

Bouchaud, Jean-Philippe, Marc Mézard, and Marc Potters. 2002. "Statistical Properties of Stock Order Books: Empirical Results and Models." *Quantitative Finance* 2 (4): 251–56. https://doi.org/10.1088/1469-7688/2/4/301.

Cao, Charles, Zhiwu Chen, and John M Griffin. 2004. "Informational Content of Option Volume Prior to Takeovers." *Journal of Business* 77 (6): 1073–1109. https://doi.org/10.1086/422629.

Cong, Fei. 2020. "Reward Engineering in Reinforcement Learning for Trading (Provisional Thesis Citation)."

Cont, Rama, Arseniy Kukanov, and Sasha Stoikov. 2014. "The Price Impact of Order Book Events." *Journal of Financial Econometrics* 12 (1): 47–88. https://doi.org/10.1093/jjfinec/nbt002.

Dempster, Michael A. H., and Yadolah Romahi. 2002. "Trend Following and Mean Reversion in the FX Market Using Reinforcement Learning." *Quantitative Finance* 2 (3): 159–76.

Deng, Yue, Feng Bao, Youyong Kong, Zhiquan Ren, and Qionghai Dai. 2017. "Deep Direct Reinforcement Learning for Financial Signal Representation and Trading." *IEEE Transactions on Neural Networks and Learning Systems* 28 (3): 653–64. https://doi.org/10.1109/TNNLS.2016.2522401.

Easley, David, Marcos M López de Prado, and Maureen O'Hara. 2012. "Flow Toxicity and Liquidity in a High-Frequency World." *Review of Financial Studies* 25 (5): 1457–93. https://doi.org/10.1093/rfs/hhs053.

Engle, Robert F. 2000. "The Econometrics of Ultra-High-Frequency Data." *Econometrica* 68 (1): 1–22. https://doi.org/10.1111/1468-0262.00091.

Fujimoto, Scott, Herke van Hoof, and David Meger. 2018. "Addressing Function Approximation Error in Actor-Critic Methods." *ICML*, 1587–96.

Glosten, Lawrence R. 1994. "Is the Electronic Open Limit Order Book Inevitable?" *Journal of Finance* 49 (4): 1127–61. https://doi.org/10.1111/j.1540-6261.1994.tb02450.x.

Glosten, Lawrence R, and Paul R Milgrom. 1985. "Bid, Ask and Transaction Prices in a Specialist Market with Heterogeneously Informed Traders." *Journal of Financial Economics* 14 (1): 71–100. https://doi.org/10.1016/0304-405X(85)90044-3.

Gold, Christian. 2003. "Policy Search Methods for Financial Trading (Provisional Thesis Citation)."

Haarnoja, Tuomas, Aurick Zhou, Pieter Abbeel, and Sergey Levine. 2018. "Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor." *ICML*, 1861–70.

Harris, Lawrence. 1986. "A Transaction Data Study of Weekly and Intradaily Patterns in Stock Returns." *Journal of Financial Economics* 16 (1): 99–117. https://doi.org/10.1016/0304-405X(86)90044-9.

———. 1991. "Stock Price Clustering and Discreteness." *Review of Financial Studies* 4 (3): 389–415. https://doi.org/10.1093/rfs/4.3.389.

Hasselt, Hado van, Arthur Guez, and David Silver. 2016. "Deep Reinforcement Learning with Double Q-Learning." In *Proceedings of the AAAI Conference on Artificial Intelligence*. Vol. 30. 1. https://doi.org/10.1609/aaai.v30i1.10295.

Jiang, Zhengyao, Dixing Xu, and Jinjun Liang. 2017. "A Deep Reinforcement Learning Framework for the Financial Portfolio Management Problem," 1–31. http://arxiv.org/abs/1706.10059.

Kabbani, Talal, and Engin Duman. 2022. "Deep Reinforcement Learning Approach for Trading Automation in the Stock Market." *IEEE Access* 10: 93564–74. https://doi.org/10.1109/ACCESS.2022.3203697.

Kingma, Diederik P., and Jimmy Ba. 2014. "Adam: A Method for Stochastic Optimization." *arXiv Preprint arXiv:1412.6980*. https://arxiv.org/abs/1412.6980.

Krauss, Christopher, Xuan Anh Do, and Nicolas Huck. 2017. "Deep Neural Networks, Gradient-Boosted Trees, Random Forests: Statistical Arbitrage on the s&p 500." *European Journal of Operational Research* 259 (2): 689–702. https://doi.org/10.1016/j.ejor.2016.10.031.

Kyle, Albert S. 1985. "Continuous Auctions and Insider Trading." *Econometrica* 53 (6):

1315–35. https://doi.org/10.2307/1913210.

Lee, Charles M C, and Mark J Ready. 1991. "Inferring Trade Direction from Intraday Data." *Journal of Finance* 46 (2): 733–46. https://doi.org/10.1111/j.1540-6261.1991.tb02683.x.

Lee, Jae Won. 2001. "Neural Network q-Learning for Stock Trading (Provisional Thesis Citation)."

Liang, Zhipeng, Haoyuan Chen, Jun Zhu, Kunyao Jiang, Yu Li, and Weinan Zhu. 2018. "Adversarial Deep Reinforcement Learning in Portfolio Management." *arXiv Preprint arXiv:1808.09940*. https://arxiv.org/abs/1808.09940.

Lillicrap, Timothy P., Jonathan J. Hunt, Alexander Pritzel, Nicolas Heess, Tom Erez, Yuval Tassa, David Silver, and Daan Wierstra. 2016. "Continuous Control with Deep Reinforcement Learning." In *International Conference on Learning Representations*.

Lin, Long-Ji. 1992. "Self-Improving Reactive Agents Based on Reinforcement Learning, Planning and Teaching." *Machine Learning* 8 (3–4): 293–321. https://doi.org/10.1007/BF00992699.

López de Prado, Marcos. 2018. *Advances in Financial Machine Learning*. Hoboken, NJ: Wiley.

Majidi, Naseh, Mahdi Shamsi, and Farokh Marvasti. 2024. "Algorithmic Trading Using Continuous Action Space Deep Reinforcement Learning." *Expert Systems with Applications* 237: 121292. https://doi.org/10.1016/j.eswa.2023.121292.

Mnih, Volodymyr, Koray Kavukcuoglu, David Silver, Andrei A. Rusu, Joel Veness, Marc G. Bellemare, Alex Graves, et al. 2015. "Human-Level Control Through Deep Reinforcement Learning." *Nature* 518 (7540): 529–33. https://doi.org/10.1038/nature14236.

Moody, John, and Matthew Saffell. 1998. "Reinforcement Learning for Trading."

———. 2001. "Learning to Trade via Direct Reinforcement." *IEEE Transactions on Neural Networks* 12 (4): 875–89. https://doi.org/10.1109/72.935097.

Mosic, Ranko. 2017. "Reinforcement Learning Based Trading Application at JP Morgan Chase." https://ranko-mosic.medium.com/reinforcement-learning-based-trading-application-at-jp-morgan-chase-f829b8ec54f2.

Neuneier, Ralph. 1998. "Enhancing q-Learning for Optimal Asset Allocation." *Advances in Neural Information Processing Systems* 10: 936–42.

Ng, Andrew Y., Daishi Harada, and Stuart Russell. 1999. "Policy Invariance Under Reward Transformations: Theory and Application to Reward Shaping." *ICML*, 278–87.

Precup, Doina, Richard S. Sutton, and Satinder Singh. 2000. "Eligibility Traces for Off-

Policy Policy Evaluation." In *ICML*, 759–66.

Puterman, Martin L. 1994. *Markov Decision Processes: Discrete Stochastic Dynamic Programming*. New York: John Wiley & Sons.

Roll, Richard. 1984. "A Simple Implicit Measure of the Effective Bid-Ask Spread in an Efficient Market." *Journal of Finance* 39 (4): 1127–39.

Rummery, Gavin A., and Mahesan Niranjan. 1994. "On-Line q-Learning Using Connectionist Systems." CUED/F-INFENG/TR 166. University of Cambridge, Department of Engineering.

Shen, Yijun. 2021. "Benchmarking Reinforcement Learning Algorithms Across Financial Markets (Provisional Thesis Citation)."

Stoikov, Sasha. 2018. "The Micro-Price: A High-Frequency Estimator of Future Prices." *Quantitative Finance* 18 (12): 1959–66. https://doi.org/10.1080/14697688.2018.1489139.

Sutton, Richard S. 1988. "Learning to Predict by the Methods of Temporal Differences." *Machine Learning* 3 (1): 9–44. https://doi.org/10.1007/BF00115009.

Sutton, Richard S., and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction*. 2nd ed. Cambridge, MA: MIT Press.

Sutton, Richard S., David McAllester, Satinder Singh, and Yishay Mansour. 2000. "Policy Gradient Methods for Reinforcement Learning with Function Approximation." *Advances in Neural Information Processing Systems* 12: 1057–63.

Tsitsiklis, John N., and Benjamin Van Roy. 1997. "An Analysis of Temporal-Difference Learning with Function Approximation." *IEEE Transactions on Automatic Control* 42 (5): 674–90. https://doi.org/10.1109/9.580874.

Wang, Zhengyao. 2019. "PPO-Based Cryptocurrency Portfolio Management (Provisional Thesis Citation)."

Watkins, Christopher J. C. H. 1989. *Learning from Delayed Rewards*. King's College, University of Cambridge.

Watkins, Christopher J. C. H., and Peter Dayan. 1992. "Q-Learning." *Machine Learning* 8 (3–4): 279–92. https://doi.org/10.1007/BF00992698.

Williams, Ronald J. 1992. "Simple Statistical Gradient-Following Algorithms for Connectionist Reinforcement Learning." *Machine Learning* 8 (3–4): 229–56. https://doi.org/10.1007/BF00992696.

Wood, Robert A., Thomas H. McInish, and J. Keith Ord. 1985. "An Investigation of Transactions Data for NYSE Stocks." *Journal of Finance* 40 (3): 723–39.

Xiong, Zhuoran, Xiao-Yang Liu, Shan Zhong, Hongyang Yang, and Anwar Walid. 2018. "Practical Deep Reinforcement Learning Approach for Stock Trading." *arXiv Preprint arXiv:1811.07522*. https://arxiv.org/abs/1811.07522.

Yang, Xiao. 2020. "Deep Reinforcement Learning for High-Frequency Trading with PPO (Provisional Thesis Citation)."

Zhang, Wei. 2020. "Attention-Based Deep Reinforcement Learning for Stock Trading (Provisional Thesis Citation)."

Zhang, Zihao, Stefan Zohren, and Stephen Roberts. 2019. "DeepLOB: Deep Convolutional Neural Networks for Limit Order Books." *IEEE Transactions on Signal Processing* 67 (11): 3001–12. https://doi.org/10.1109/TSP.2019.2907260.

A. LOB Feature Inventory

The table below is the complete specification of all features used in the state-space experiments. Feature names match the keys in the feature registry (`src/trading_rl/features/lob_features.py`) the active set is defined in `src/configs/features/hft_lob_features_state_space.yaml`. Formulas are given in Chapter 5. All features except the hour encodings are z-score normalized on the training split only.

Feature name	**Group**	**Param**	**Literature**
`hft_book_pressure_l0`	Imbalance	level 0	Cao et al. (2004); Biais et al. (1995)
`hft_book_pressure_l1`	Imbalance	level 1	Cao et al. (2004)
`hft_book_pressure_l2`	Imbalance	level 2	Cao et al. (2004)
`hft_order_book_imbalance_3l`	Imbalance	$N = 3$	Cont et al. (2014)
`hft_order_count_imbalance_l0`	Imbalance	level 0	Biais et al. (1995)
`hft_microprice`	Fair value	—	Stoikov (2018)
`hft_microprice_divergence`	Fair value	—	Stoikov (2018)
`hft_vwmp_skew`	Fair value	$N = 3$	Stoikov (2018)
`hft_spread_bps`	Spread/cost	—	Kyle (1985); Glosten & Milgrom (1985)
`hft_spread_ratio_50`	Spread/cost	$W = 50$	Easley et al. (2012)
`hft_bid_convexity`	Spread/cost	bid side	Bouchaud et al. (2002)
`hft_ask_convexity`	Spread/cost	ask side	Bouchaud et al. (2002)

(continued on next page)

(continued from previous page)

Feature name	Group	Param	Literature
`hft_bid_slope`	Spread/cost	5 levels	Bouchaud et al. (2002); Kyle (1985)
`hft_ask_slope`	Spread/cost	5 levels	Bouchaud et al. (2002); Kyle (1985)
`hft_ofi`	Order flow	—	Cont et al. (2014)
`hft_ofi_rolling_50`	Order flow	$W = 50$	Cont et al. (2014)
`hft_bid_queue_depletion`	Order flow	bid side	Foucault et al. (2005)
`hft_ask_queue_depletion`	Order flow	ask side	Foucault et al. (2005)
`hft_signed_trade_flow_50`	Order flow	$W = 50$	Lee & Ready (1991)
`hft_odd_lot_trade_ratio_200`	Order flow	$W = 200$	Boehmer et al. (2021)
`hft_odd_lot_imbalance_200`	Order flow	$W = 200$	Boehmer et al. (2021)
`hft_inter_event_time`	Regime/temp.	—	Engle (2000)
`hft_mid_price_acceleration`	Regime/temp.	—	Abergel et al. (2016)
`hft_hour_sin`	Regime/temp.	—	Wood et al. (1985)
`hft_hour_cos`	Regime/temp.	—	Wood et al. (1985)

B. TD3 Hyperparameters

The table below lists all hyperparameters for the main AAPL state-space experiment. The full configuration file is `src/configs/scenarios/aapl_td3_hft_lob_state_space.yaml`.

Parameter	Value	Notes
Architecture		
Actor hidden dims	[128, 64]	ReLU activations
Critic hidden dims	[128, 64]	ReLU activations
Optimization		
Actor learning rate	0.0001	Adam
Critic learning rate	0.0001	Adam
Actor weight decay	0.000002	L2 regularization
Critic weight decay	0.000002	
Loss function	Smooth L1 (Huber)	

(continued on next page)

(continued from previous page)

Parameter	Value	Notes
TD3 mechanisms		
Discount factor γ	0.99	
Target update τ	0.001	Soft update
Policy delay	2	Critic updates per actor update
Target policy noise σ	0.2	Added to target action
Noise clip	0.3	
Exploration noise std	0.2	During training only
Training		
Total steps	2,000,000	
Initial random steps	1,000	Warm-up before learning
Frames per batch	200	
Optimisation steps per batch	5	
Mini-batch size	128	
Replay buffer size	100,000	
Data		
Training rows	15,000	Chronological split
Reward type	Differential Sharpe	$\eta = 0.01$
Transaction fee	0.000	Baseline setting
Random seed	42	

C. Experiment Configuration Example

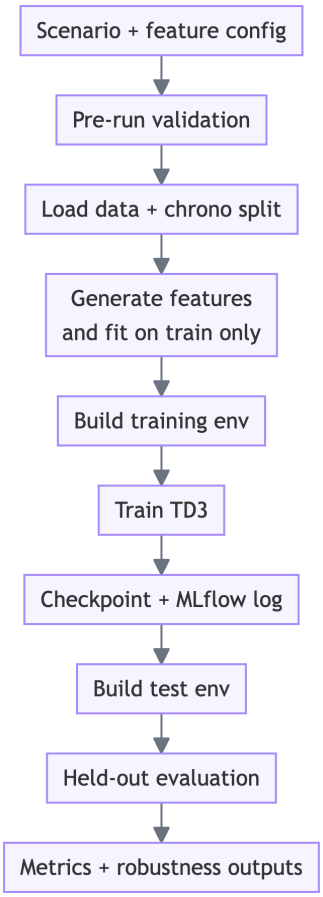
The YAML snippet below is a condensed version of the main experiment configuration, illustrating the configuration-driven workflow described in Chapter 6.

```yaml
seed: 42


data:
  data_path: "./data/raw/stocks/AAPL_2026-02-25_2026-02-27_raw_mbp-10_us_hours.parquet"
```

```yaml
  feature_config: "src/configs/features/hft_lob_features_state_space.yaml"
  train_size: 15000

env:
  backend: "tradingenv"
  mode: "hft"
  trading_fees: 0.000
  price_column: "close"
  reward_type: "differential_sharpe"
  reward_eta: 0.01

network:
  actor_hidden_dims: [128, 64]
  value_hidden_dims: [128, 64]

training:
  algorithm: "TD3"
  actor_lr: 0.0001
  value_lr: 0.0001
  gamma: 0.99
  tau: 0.001
  policy_noise: 0.2
  noise_clip: 0.3
  policy_delay: 2
  max_steps: 2000000
  sample_size: 128
  buffer_size: 100000
  loss_function: "smooth_l1"
```

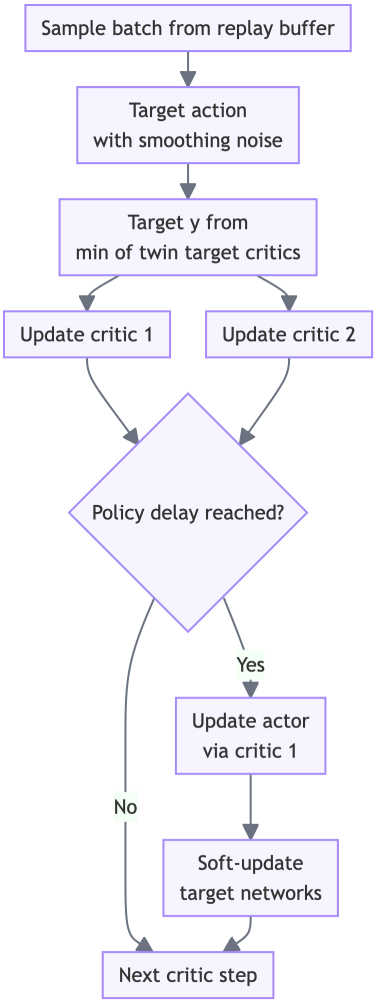
D. TD3 Training Workflow Diagram

The diagram below summarizes the configuration-driven workflow described in Chapter 6.



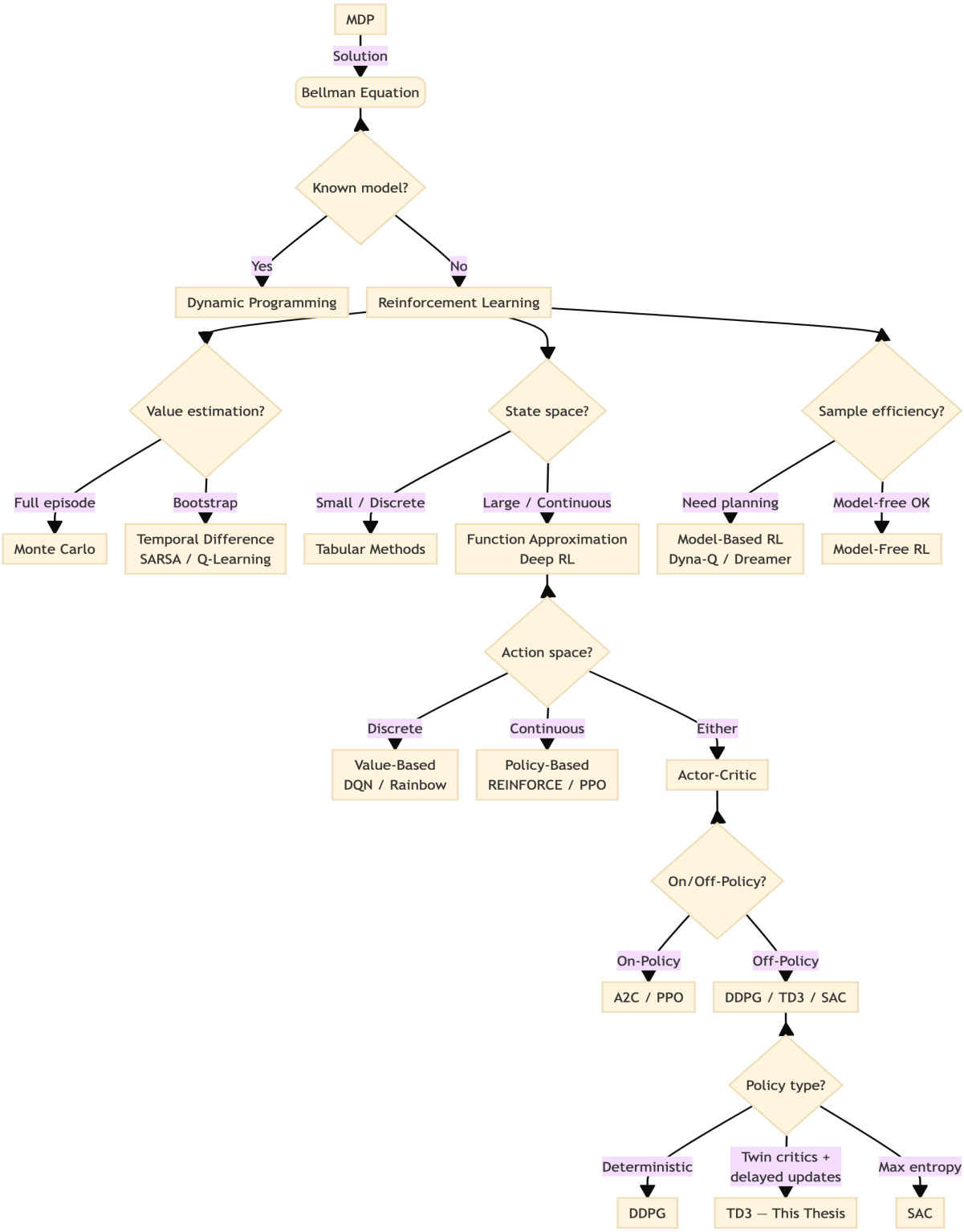
E. TD3 Update Mechanics Diagram

The diagram below summarizes the core TD3 update mechanics used in the thesis implementation, abstracting away from project-specific plumbing.



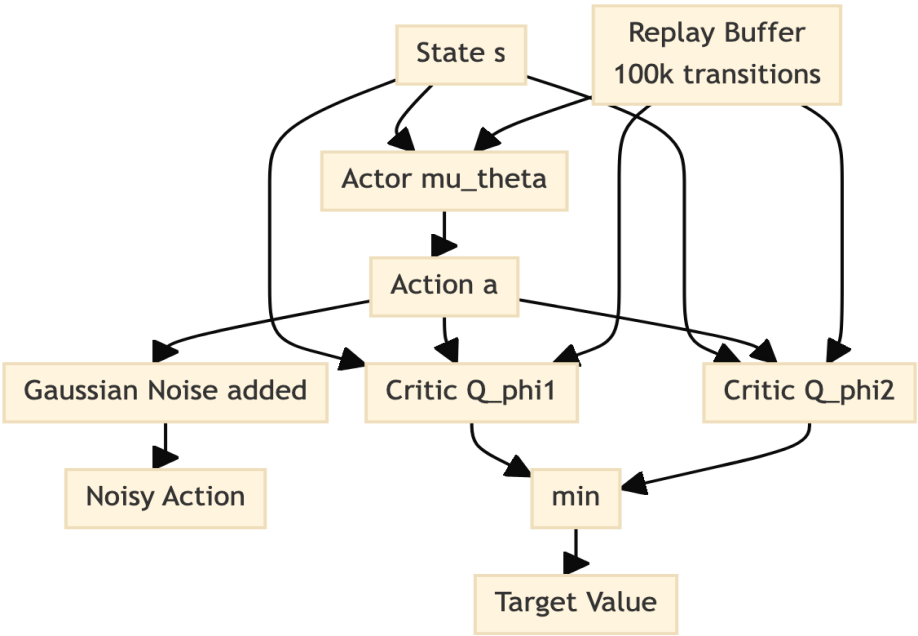
F. RL Algorithm Taxonomy

The diagram below maps the reinforcement learning landscape from the Bellman equation through the key algorithmic relaxations, showing where TD3 sits relative to other families of methods.


MDP
Solution
Bellman Equation
Known model?
Yes
No
Dynamic Programming
Reinforcement Learning
Value estimation?
State space?
Sample efficiency?
Full episode
Bootstrap
Small / Discrete
Large / Continuous
Need planning
Model-free OK
Monte Carlo
Temporal Difference
SARSA / Q-Learning
Tabular Methods
Function Approximation
Deep RL
Model-Based RL
Dyna-Q / Dreamer
Model-Free RL
Action space?
Discrete
Continuous
Either
Value-Based
DQN / Rainbow
Policy-Based
REINFORCE / PPO
Actor-Critic
On/Off-Policy?
On-Policy
Off-Policy
A2C / PPO
DDPG / TD3 / SAC
Policy type?
Deterministic
Twin critics +
delayed updates
Max entropy
DDPG
TD3 — This Thesis
SAC

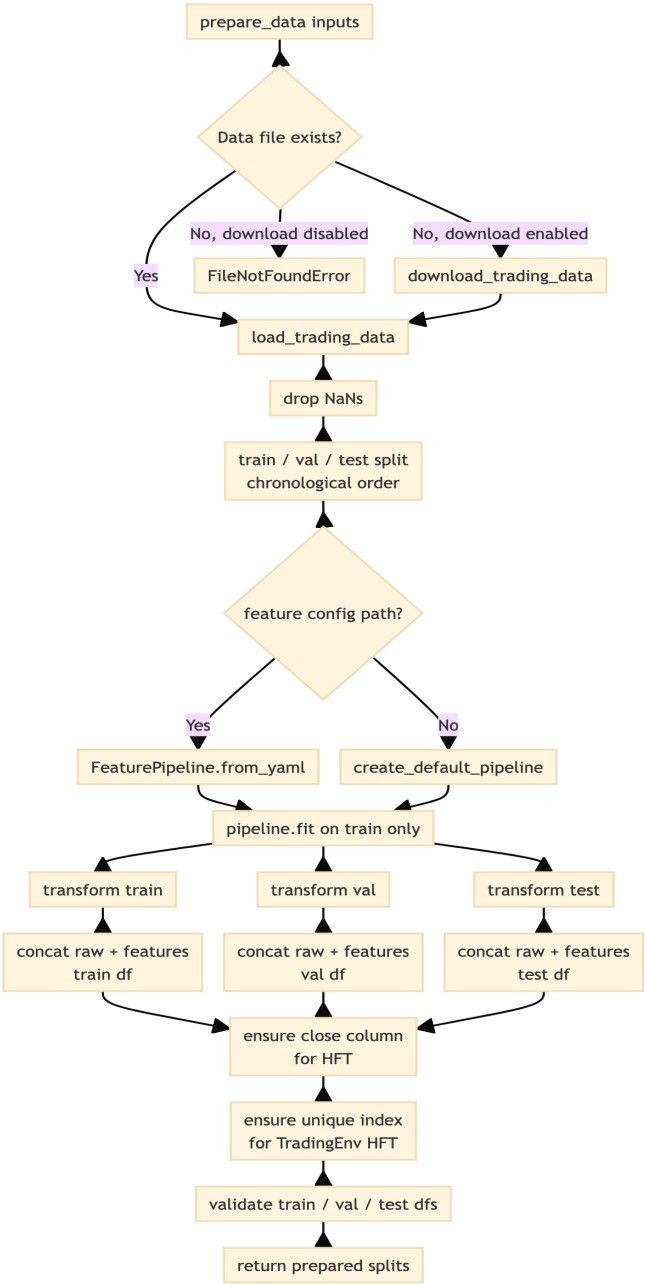
G. TD3 Network Architecture

The diagram below shows how the actor, twin critics, and replay buffer interact in the TD3 architecture used in this thesis.



H. Data Preparation Pipeline

The diagram below shows the feature pipeline design that ensures no information from the validation or test splits leaks into the fitted normalization parameters.



I. TD3 Training Pseudo-Code

Algorithm 1 TD3 Trading Experiment Workflow

Require: Experiment configuration (data path, feature set, hyperparameters)
Ensure: Trained policy μ_θ, evaluation metrics

```
Load and validate configuration; check paths and split feasibility
Load AAPL MBP-10 dataset; sort chronologically
Split data into train / validation / test subsets (chronological)
Fit feature pipeline on training split only
Transform all splits using fitted pipeline
Build training environment from processed training data
Initialise actor μ_θ, twin critics Q_{φ_1}, Q_{φ_2}, target networks μ_{θ'}, Q_{φ'_1}, Q_{φ'_2}, replay buffer D
for each step t = 1, ..., max_steps do
    a_t ← μ_θ(s_t) + ε,    ε ~ N(0, σ_explore)
    Execute a_t; observe r_t, s_{t+1}; store (s_t, a_t, r_t, s_{t+1}) in D
    Sample mini-batch B ~ D
    Compute smoothed target action: ã' ← μ_{θ'}(s') + clip(N(0, σ_noise), ±c)
    Compute Bellman target: y ← r + γ · min_{i=1,2} Q_{φ'_i}(s', ã')
    Update both critics by minimising L_{Smooth-L1}(y − Q_{φ_i}(s, a))
    if t mod policy_delay = 0 then
        Update actor: θ ← θ + α∇_θ Q_{φ_1}(s, μ_θ(s))
        Soft-update targets: φ'_i ← τφ_i + (1 − τ)φ'_i;   θ' ← τθ + (1 − τ)θ'
    end if
    Checkpoint and log metrics at configured intervals
end for
Evaluate deterministic policy μ_θ on test split
Compute financial performance metrics from realised return series
```